



面向 AI 智能体的操作系统内核

AgentOS-uCore: 基于 RISC-V uCore 的系统原生智能体运行机制

项目名称	面向 AI 智能体的操作系统内核
队伍名称	happy-legend
指导老师	余盛季、郭力维
目标平台	RISC-V 64 / uCore / QEMU
文档版本	决赛设计开发文档

姓名	联系方式 (QQ)
王浩泮	2091576055
康俊豪	488718235

2026 年 08 月

摘 要

智能体工作流已经从单轮问答发展为持续的工具调用、文件处理和多角色协作。现有应用层框架能够安排模型和工具，却往往只在应用日志里保存角色、工作流代次、上下文因果关系和资源归属。当工具真正写入文件、发送消息或申请 CPU 服务时，内核看到的仍是普通进程、文件和 IPC，无法据此作出一致判断。

本项目名称为“面向 AI 智能体的操作系统内核”，基于 LearningOS/uCore 实现 AgentOS-uCore。系统沿用 RISC-V uCore 的进程、虚拟内存、VFS、系统调用和调度器，并把 Agent（智能体）登记为内核能够识别的工作负载。Agent 进程创建与地址空间设计把低频的身份、配额和生命周期状态留在 PCB，将高频读取的 Context 镜像和用户缓存映射到固定用户地址；Agent-OS 内核结构化交互接口以 26 项工具目录、schema（参数模式）、执行约定（Execution Contract）和任务通道承接工具请求；上下文路径、来源追溯（provenance）和执行记录保存请求、结果与数据来源；文件查询、事件等待和工作流 EEVDF 则分别处理属性检索、Agent Loop 唤醒和 CPU 服务分配。

在集成层，AgentOS 控制台按固定步骤执行科研恢复工作流；AgentOS Nexus 则在同一次 QEMU 启动中持续运行协调、系统和研究三个业务角色。Nexus 的 child Task 由 Coordinator 经原生 delegated Task Channel 委派，System 或 Research 领取后把结果写为 Guest artifact（任务结果文件），Coordinator 从唯一 terminal CQE 结算。工作区工具先在 Guest 的 Metadata Catalog 中登记 Host 提供的版本化 manifest，再由 Live Query 选择候选，并用 Typed Watch 处理 generation 变化；Host 在已选候选中匹配正文或返回指定 revision 的字节。真实结果回到 Guest 后进入 Research artifact 与 Context，跨轮消息由 Guest Context active path 重建。

性能测量批次对应源码提交 2b14fb，使用 30 次相互独立的全新 QEMU 启动，保留 33 份原始串口输出、19 份 CSV 数据表和 7,498 条结构化记录。在 16 组独立工作流配对中，带索引核心查询的耗时均低于遍历查询，配对加速比中位数为 **3.118 倍**；在同样 16 组配对的端到端窗口中，索引路径仅有 **3/16 组** 更快，其耗时减去遍历路径的中位差值为 **+13.452 ms**，说明核心查询之外的准备与收尾开销尚未得到控制。文件查询网格覆盖目录规模与命中数的 12 种组合，任务通道保留 96 条包含 16 个操作的序列；EEVDF 的 504 条唤醒探针均在 0–1 个时钟节拍（tick）内获得服务。

设计主题	内核机制	可观察行为
Agent 进程创建与地址空间设计	PCB 扩展、受信任映像、七页 Agent Context 区	普通进程与 Agent 进程共存；Guest 可直接读取六页内核镜像，并使用一页用户缓存。
Agent-OS 内核结构化交互接口	工具目录、V2/V3 请求和固定大小返回结构	工具可发现，参数按 schema 解码；未知工具、类型错误和权限不足均返回明确状态。
工具调用协议	Execution Contract、Batch、任务通道 SQ/CQ、原生跨 Agent 委派	同步短调用、带约定调用和有界队列共用一套工具语义；child Task 通过 claim/complete、协作式终态确认和唯一 terminal CQE 结算。
上下文路径管理	128 条有界路径、只读镜像、分支和回滚	连续调用形成可直接读取的 active path；Nexus 从该路径重建 USER、TOOL 与成功 FINAL，失败或取消时回滚。
面向 Agent 查询优化的文件系统扩展	Metadata Catalog、选择性索引、Live Query 和 Typed Watch	Agent 按属性选择 Host manifest 候选；Host 只返回候选的版本绑定字节，正文再作为 Guest artifact 和 Context 使用。
Agent Loop 内核运行机制	心跳、事件等待、消息路由、工作流 EEVDF	无事件时线程休眠，消息或 TIMER 到达后唤醒；多个工作流按实体而非线程数获得服务。
综合场景	恢复控制台和 Nexus 三角色协作	同一 RV64 Guest 中完成 Context 驱动的多轮交互、原生 Task Channel 委派、版本化工作区读取与会话关闭。

表 1: AgentOS-uCore 的系统能力与可观察行为

目录

摘要	ii
第一章 项目背景	1
1.1 行业背景与问题定义	1
1.1.1 从内容生成到可执行副作用	1
1.1.2 智能体执行的基础对象与风险链条	2
1.1.3 现有方案的特性与职责边界	2
1.1.4 AgentOS-uCore 的系统方案	3
1.2 核心挑战与设计目标	4
第二章 系统设计	5
2.1 AgentOS-uCore 总体架构	5
2.1.1 以 uCore 为底座的产品分层	5
2.1.2 六项核心机制与请求流向	6
2.1.3 一次智能体工具调用	6
2.1.4 上层 runtime	6
2.2 跨模块状态约束	7
第三章 系统实现	8
3.1 Agent 进程创建与地址空间设计	8
3.1.1 从普通进程到 workflow 中的 Agent	8
3.1.2 身份字段与角色策略	9
3.1.3 Agent Context 区的七页布局	9
3.1.4 生命周期代次与并发控制	10
3.1.5 随 workflow 创建的资源主体	10
3.1.6 验证结果	10
3.2 Agent-OS 内核结构化交互接口	10
3.2.1 工具调用协议与工具目录	10
3.2.2 自适应的 V2 调用	11
3.2.3 四种执行路径	11
3.2.4 工具执行期间的资源使用租约	11
3.2.5 带类型信息的任务通道	12
3.2.6 原生 delegated Task Channel	12
3.2.7 验证结果	13
3.3 上下文路径管理、数据来源与 Workflow Fence	13
3.3.1 从多轮调用到 Context Path	13
3.3.2 分支、回滚与有界淘汰	14
3.3.3 数据来源传播	14
3.3.4 工具清单与副作用检查	14
3.3.5 执行记录环	14
3.3.6 Workflow Fence	15
3.3.7 验证结果	15
3.4 面向 Agent 查询优化的文件系统扩展	15
3.4.1 从路径访问到属性查询	15
3.4.2 元数据与 inode 代次	15
3.4.3 遍历查询与选择性索引	16
3.4.4 带类型信息的实时订阅与缺口重新同步	16
3.4.5 查询性能	16
3.4.6 验证结果	17
3.5 Agent Loop 内核运行机制	18
3.5.1 从轮询循环到内核等待	18
3.5.2 心跳与模型请求关联号	18
3.5.3 工作流资源账户	18
3.5.4 按工作流汇总的 EEVDF	19
3.5.5 唤醒延迟与公平性	19
3.5.6 验证结果	19
3.6 AgentOS 控制台综合工作流	20
3.6.1 科研恢复场景	20
3.6.2 遍历与索引路径的配对实验	20
3.6.3 核心查询与完整流程窗口	20
3.6.4 系统结果	21
3.7 综合场景：AgentOS Nexus 多智能体协作平台	21
3.7.1 长驻会话与三角色协作	21
3.7.2 一次交互回合	22
3.7.3 原生 delegated Task Channel	23
3.7.4 Guest Catalog 与 Host 工作区边界	23

3.7.5	artifact、来源与 Context	24
3.7.6	跨轮 Context 路径	24
3.7.7	工具发现与内核观测	24
3.7.8	运行验证	25
3.8	实现演进与核心片段	25
3.8.1	身份代次：从访问范围到生命周期键	25
3.8.2	结构化工具：从字段校验到 Execution Contract	26
3.8.3	Context：从 FIFO 截短到 active path 与 provenance	27
3.8.4	Catalog：从自动扫描到 Typed Watch	29
3.8.5	任务委派：从 N1 MESSAGE 到 native Task Channel	30
3.8.6	工作流服务：从线程份额到 Credit Domain 与 EEVDF	32
3.8.7	Nexus 边界：Host Broker 与 Guest Authority	33
第四章	项目测试	35
4.1	功能与性能测试	35
4.1.1	测试组织	35
4.1.2	任务一至任务五的 Guest 综合验收	36
4.1.3	结构化工具与任务通道测量	36
4.1.4	结果文件发布与冲突处理	37
4.1.5	文件查询路径的 I/O 观测	37
4.1.6	测量数据与可重复性	38
4.2	测试路径与结果	38
4.2.1	Guest 原生工作负载	39
4.2.2	工具、执行约定与任务通道测试	40
4.2.3	Nexus 会话与 Host Relay	41
4.2.4	失效状态与恢复分支	41
第五章	总结与展望	42
5.1	阶段性结论	42
5.2	当前技术限制	42
5.3	后续工作	42
5.4	比赛收获	42
第六章	参考文献	44

1. 项目背景

当智能体能够创建进程、读写文件、调用工具并委派子任务时，模型侧的风险会转化为可观察的系统副作用。身份、任务、上下文、文件和资源因而需要在执行层共享一致的生命周期与权限规则，并留下能够复盘的状态记录。

1.1. 行业背景与问题定义

1.1.1. 从内容生成到可执行副作用

大模型应用正在从“生成一段内容”转向“围绕目标持续行动”。典型智能体会读取上下文、规划步骤、选择工具、等待结果并修正计划，还可能把子任务委派给其他智能体。此时，模型输出不再停留在文本窗口，而会驱动进程创建、文件读写、消息发送、外部服务调用和资源消耗。一次错误判断或受污染的上下文，因而可能沿“模型决策-工具调用-系统副作用-新上下文”的闭环被放大；仅在提示词或应用日志中记录角色和权限，难以保证实际执行路径采用同一套身份、生命周期和资源规则。

NIST 发布的《Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile》（NIST AI 600-1）指出，生成式 AI 风险可能出现在设计、开发、部署、运行及退役等阶段，也可能位于模型、应用乃至生态层面；该报告围绕治理、内容来源、部署前测试和事件披露等重点，给出 Govern、Map、Measure、Manage 框架下的建议行动 [1]。从操作系统视角看，这意味着执行层不仅要“把调用做完”，还应留下稳定、可核对的身份、来源、状态迁移和资源证据，使上层的测试、监测和事件复盘有可信落点。

OWASP《Top 10 for LLM Applications 2025》将提示注入、敏感信息泄露、不当输出处理、过度代理能力和无界资源消耗等列为 LLM 应用需要优先关注的风险类别 [2]。图 1.1 在官方总览图上标出了与本项目执行层关系最直接的五项。

OWASP Top 10 for LLM Applications 2025

Official report contents, Version 2025

LLM01:2025 Prompt Injection	3	LLM06:2025 Excessive Agency	22
LLM02:2025 Sensitive Information Disclosure	7	LLM07:2025 System Prompt Leakage	26
LLM03:2025 Supply Chain	11	LLM08:2025 Vector and Embedding Weaknesses	29
LLM04: Data and Model Poisoning	16	LLM09:2025 Misinformation	32
LLM05:2025 Improper Output Handling	19	LLM10:2025 Unbounded Consumption	35

Source: OWASP GenAI Security Project, OWASP Top 10 for LLM Applications 2025, Version 2025 (18 November 2024). Red frames are annotations added for this document.

图 1.1: OWASP Top 10 for LLM Applications 2025 及本文关注的执行层风险

红框所示五项为 LLM01 Prompt Injection（提示注入）、LLM02 Sensitive Information Disclosure（敏感信息泄露）、LLM05 Improper Output Handling（不当输出处理）、LLM06 Excessive Agency（过度代理能力）和 LLM10 Unbounded Consumption（无界资源消耗）。原图来自 OWASP Gen AI Security Project[2]，本文依 CC BY-SA 4.0 对其作标注。

进一步地，OWASP《Agentic AI – Threats and Mitigations》v1.1 将智能体的规划与推理、记忆与状态、工具使用以及单智能体和多智能体交互放入同一参考架构，并分别讨论身份、

工具、记忆和协作链路中的威胁与缓解思路 [3]。这些风险延伸到实际执行层：即使模型服务本身不变，权限继承、任务取消、结果归属、上下文来源和资源结算仍需要稳定的系统机制。

1.1.2. 智能体执行的基础对象与风险链条

从系统实现看，Agent Loop（智能体循环）将若干传统对象组织成更长的因果链：模型给出决策，用户态运行时把决策翻译为结构化请求，内核和设备执行副作用，结果再成为下一轮上下文。为了约束和复盘这一链条，至少需要区分表 1.1 中的四类对象。

基础对象	需要回答的问题	只留在应用文本中的风险
身份与权限	当前行动属于哪个智能体、角色和工作流代次，能够访问哪些对象、调用哪些工具	标识符复用后误认旧主体；子进程或代理链权限漂移；不同入口采用不同授权判断
工具请求与副作用	请求使用何种参数结构、由谁执行、结果对应哪次请求，何时算作提交	松散文本在各层重复解析；错误结果与新请求错配；重试导致副作用重复发生
上下文与来源	本轮决策读取了什么、数据来自哪里、文件及工作区处于哪个版本	过期或受污染状态被持续复用；结果与输入版本脱节；故障后无法重建因果链
任务与资源	子任务由谁委派、处于何种状态、取消或终态如何传播，工作流消耗多少资源	任务重复认领或完成；取消停留在消息层；通过增加线程放大调度份额

表 1.1: 智能体执行链上的基础对象

其中，能力（capability）表示内核允许主体执行的一组动作，代次（generation）用于区分同一数值标识在不同时期代表的主体，来源追溯（provenance）记录数据和结果的因果来源，终态（terminal state）则约束完成、失败和取消按照明确状态机发生。它们共同回答“谁在什么版本上，以何种权限完成了哪次可观察动作”；内容真伪和自然语言推理仍由上层判断。

1.1.3. 现有方案的特性与职责边界

业界已经形成多类智能体基础设施。表 1.2 从系统执行层比较其主要特性和适用范围。

方案类别	主要特性	用于系统执行时的边界
应用层智能体编排器	提供状态图、记忆、重试、人工确认和多智能体协作，便于快速实现业务流程	策略通常依赖单个运行时；当动作跨越进程、系统调用或不同工具服务时，身份和终态容易被多份状态分别维护
MCP、A2A 等互操作协议	以结构化消息描述工具、资源或智能体通信，降低框架与服务之间的接入成本 [4, 5]	协议定义通信契约，但不替本地操作系统裁定进程权限、VFS 访问、任务代次、取消后的资源回收和 CPU 公平性
容器、沙箱、ACL 与 cgroup	提供成熟的进程隔离、访问控制和粗粒度资源上限，适合划分服务或租户边界	通常以进程、容器或用户为主体，难直接表达一次工具调用的 schema、智能体角色代次、上下文来源及跨进程工作流的统一结算
数据库、向量库与 workflow 引擎	提供持久化、检索、检查点和失败重试，适合保存业务状态和长期记忆	需要把进程生命期、文件版本和任务终态复制到外部存储；崩溃清理、版本绑定和跨存储因果核对仍需额外协议

表 1.2: 现有方案的能力与执行层边界

应用层负责模型编排、业务策略、持久化知识库和网络协议，操作系统执行层则统一管理进程权限、工具参数契约、任务代次、上下文因果关系和资源结算。由上层决定“做什么”，由公共执行层稳定回答“谁被允许做、动作属于哪次任务、造成了什么副作用、消耗了多少资源”。

1.1.4. AgentOS-uCore 的系统方案

AgentOS-uCore 建立在 LearningOS/uCore 教学内核之上 [6]，保留 RISC-V 启动、普通进程、虚拟内存、VFS、异常处理和系统调用主干，在现有 PCB、VFS 对象、IPC 与调度路径中补充智能体身份、上下文、结构化工具调用、任务通道、来源记录、资源账户和工作流调度实体。新接口使用定长且带版本和结构大小的 UAPI；普通 uCore 进程仍走原有路径，智能体进程则通过受控的创建和 exec 路径取得内核身份。

AgentOS-uCore 为图 1.1 中的五项风险提供执行层支撑：对 LLM01，内核在模型决策形成后限制可调用工具和可提交副作用；对 LLM02，身份、能力和文件作用域约束访问并记录来源；对 LLM05，结构化参数、返回约定和请求关联号阻止松散文本直接驱动系统调用；对 LLM06，内核身份、能力位、Task Channel 状态机和 route 检查约束委派与执行；对 LLM10，工作流资源账户、原子等待和 EEVDF 调度记录用量并分配服务。提示语义、数据敏感性和业务语义仍由上层判断。

项目把多个应用都会重复实现、又必须在真实副作用发生处统一裁定的机制下沉到 uCore，在执行层提供最小权限、明确终态、可追溯证据和有界资源。模型服务、传输加密和密钥管理继续由 Host 承担，应用层负责模型输出与业务语义。

1.2. 核心挑战与设计目标

AgentOS-uCore 的核心挑战不是增加若干彼此独立的系统调用，而是让身份、任务、上下文、文件和调度在同一条执行链上对“主体与代次”给出一致答案。项目同时保留普通 uCore 工作负载的原有语义，将文件选择与跨轮历史留在 Guest，并用 Guest 行为、Host 记录 and 固定工作负载共同验证。表 1.3 将这些要求整理为“挑战-设计目标-验证路径”。

核心挑战	设计目标	验证路径
身份与权限跨对象保持一致	以 id+generation 作为内核主体，角色、能力位和控制标识随受控的 create、fork、exec、exit 路径迁移；工具、VFS、IPC 与任务路由使用同一身份判定	同时运行普通进程与智能体进程；覆盖创建、继承、映像替换、退出和标识复用；检查越权、过期代次和错误 route 均被拒绝
任务委派具有唯一、可取消的终态	由原生 Task Channel 的 SQ/CQ 和内核状态机维护 submit、claim、complete、cancel 及背压，避免把终态仅作为应用消息约定	构造并发认领、重复完成、取消竞态、队列满和过期请求；核对状态码、完成项及资源回收结果
上下文、来源与工作区版本可以复盘	以七页 Context 区承载有界活动路径，以 dev+inum+incarnation 绑定启动期文件元数据、索引和订阅；Host 只在显式 root 边界内提供 manifest 与版本字节，不替 Guest 选择文件或改写 Provider 历史	对比遍历、属性查询和索引结果；触发增量缺口后的重同步；检查 artifact、Context、manifest 与模型请求记录的版本和关联号
资源结算抵抗线程扩张并支持事件驱动	以工作流为资源与调度实体，汇总 CPU 等资源，用原子等待替代轮询，并按 EEVDF 的滞后量和虚拟截止时间分配服务	比较单线程与多线程工作流的服务份额；验证无事件时休眠、事件到达后唤醒；报告等待时延、吞吐与资源账户
扩展保持兼容且证据可重复	保留普通 uCore 路径；所有新增 UAPI 使用定长、版本化结构；功能与性能样本绑定相同输入、工作区清单和结果指纹	运行基础回归、静态检查与 QEMU Guest 测试；使用固定回放核对 Host/Guest 证据；性能比较拒绝指纹不一致的样本

表 1.3: AgentOS-uCore 的核心挑战、设计目标与验证路径

这些目标最终落实到三类可检查对象：内核保存的状态、触发状态变化的调用路径，以及覆盖正常与异常分支的测试结果。后续章节依次展开 Host、Guest 与内核的职责分工，并说明身份、Context、结构化工具、任务通道、VFS 查询、IPC、资源和调度的实现与验证。

2. 系统设计

AgentOS-uCore 沿用 uCore 的进程、页表、VFS、异常处理、系统调用和调度主干，并直接扩展现有内核对象。一个智能体发起工具调用时，身份检查、上下文记录、文件访问、消息投递、资源记账和调度选择都读取进程中保存的同一组 workflows 编号和代次。图示、数据结构和状态机明确规定这些模块何时读取字段、何时更新状态，避免同一请求在不同位置被认作不同 workflow。

2.1. AgentOS-uCore 总体架构

2.1.1. 以 uCore 为底座的产品分层

AgentOS-uCore 按部署职责分为五部分：uCore 基础内核、AgentOS 内核模块、Guest Runtime、Host Relay 和 Agent 应用。图2.1自下而上展示这五部分：uCore 提供进程、内存、VFS、系统调用和调度器等基础对象；AgentOS 在这些对象上补充智能体身份、工具、Context、文件查询、Loop 和工作流调度；Guest Runtime 封装统一 UAPI，Host Relay 连接串口、TLS 与模型 Provider，控制台、Nexus 与科研工作流则负责具体任务。

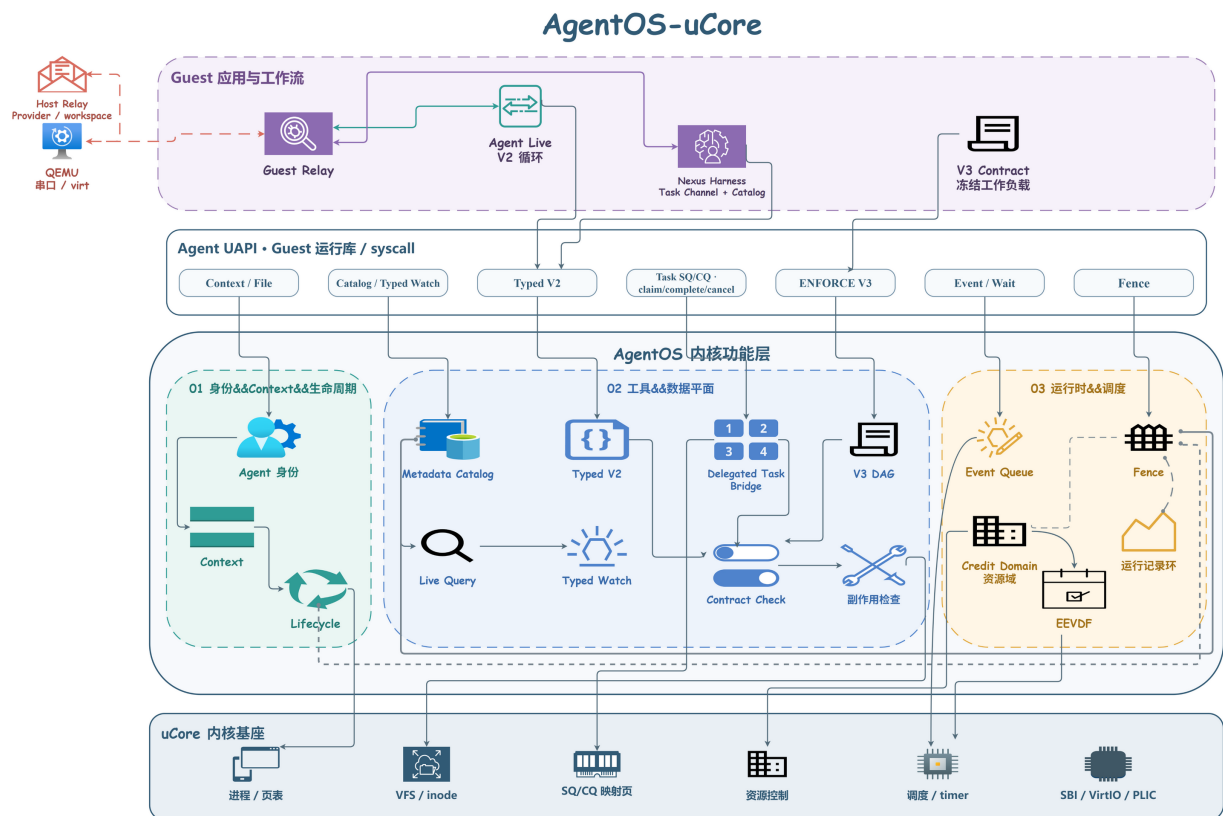


图 2.1: AgentOS-uCore 总体架构

底层 uCore 主干继续承担进程、页表、VFS、异常处理、系统调用和调度器的原有职责。AgentOS 在此基础上把相关机制分为三组：

1. 身份、上下文和资源：身份与生命周期登记智能体；上下文保存每次工具交互；资源

账户和 EEVDF 分别记录资源使用并分配 CPU 服务。

2. **工具、文件和消息：**带类型信息的工具调用检查 schema 和能力位；VFS 元数据与实时查询处理文件对象；消息路由传递跨智能体消息。
3. **来源记录和工作流结束：**上下文路径记录因果关系；来源追溯随文件、工具和 IPC 传播；执行记录环与 Workflow Fence（工作流栅栏）分别保存阶段记录和结束时的快照。

2.1.2. 六项核心机制与请求流向

模块	关键对象	主要功能
Agent 进程与地址空间	身份记录、七页 Context 区、控制标识、生命周期	创建受信任 Agent，绑定角色、能力位和工作流代次，并把高频状态映射给 Guest
结构化交互与工具协议	工具目录、V2/V3、SQ/CQ	发现工具、检查带类型信息的参数，并支持动态调用、冻结执行约定和有界任务提交
上下文路径与来源追溯	路径镜像、分支、执行记录环	连接请求、结果、前驱记录和数据来源，并按配额淘汰旧记录
面向 Agent 的文件查询	元数据、摘要、索引、订阅器	为显式登记的文件提供属性查询、结构化结果、变更通知和缺口重新同步
Agent Loop	事件队列、路由、心跳	提供原子等待与唤醒、消息、模型请求关联、超时和取消
工作流资源与调度	资源账户、EEVDF 实体	按工作流汇总资源记账，并根据滞后量和虚拟截止时间分配 CPU 服务

表 2.1: AgentOS 的核心模块

六项机制使用同一个工作流 id+generation。一次工具调用会同时取得调用者身份、前一条上下文记录、schema、数据来源、资源账户和调度实体。工作流代次变化后，已经回收槽位对应的订阅器和句柄不能再指向新对象。

2.1.3. 一次智能体工具调用

一次完整调用从 Guest 读取工具目录开始。Guest 按角色列出可用工具，再将模型结果编码为带类型信息的请求。请求进入内核后，AgentOS 依次拷入数据，检查版本和大小，检查身份与生命周期，解析工具并解码 schema，再检查执行约定、数据来源、资源和副作用，最后调用 VFS、IPC 或其他 uCore 子系统。返回前，上下文路径写入一条新记录，下一轮可引用其序号。

当智能体等待模型、文件事件或其他角色时，agent_wait 将线程置于等待状态。事件入队与等待者安装按原子顺序执行，从而避免“发现队列为空后恰好错过唤醒”。

2.1.4. 上层 runtime

控制台按固定策略依次执行身份检查、上下文记录、文件查询、IPC 和恢复操作，用于功能演示和配对测量。Nexus 使用同一组系统调用维持会话、分派任务、整理结果文件并连接模型服务。两者访问同一批 AgentOS 内核对象，因此确定性测试和自然语言交互遵循相同的系统规则。

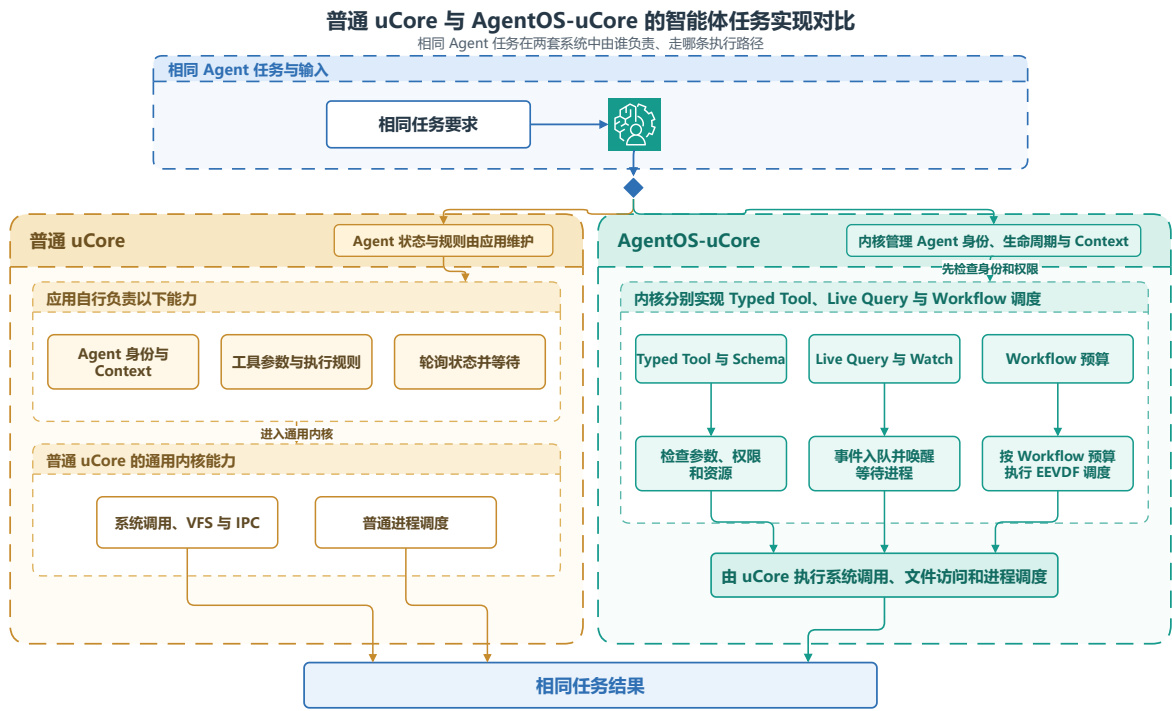


图 2.2: 同一科研恢复任务在普通 uCore 与 AgentOS-uCore 中的执行路径

在相同业务步骤下，普通 uCore 可以用进程、文件和 IPC 串起科研恢复任务，角色、上下文和等待关系由应用自行维护；AgentOS-uCore 则加入可识别的工作流标识、结构化工具、选择性索引、事件等待和工作流调度，使身份检查、数据来源和资源归属贯穿每个阶段。

2.2. 跨模块状态约束

系统将工作流标识写为可复用槽位的 id+generation。模块使用该标识前，先确认对象仍在运行、访问范围相同且代次匹配；资源、元数据和执行记录也按该标识归属。这样，旧槽位关闭、回收并再次使用后，先前的请求不会被新工作流接收。

算法 1：工作流标识的跨模块校验

输入：调用进程、目标对象、请求携带的工作流编号和代次、访问范围。

输出：可继续执行的对象引用，或明确的 STALE、DENIED 结果。

1. 从调用进程读取内核保存的工作流编号、代次和能力位。
2. 检查工作流编号和代次的结构，确认生命周期仍处于活跃状态并且代次匹配。
3. 比对调用者、目标对象和请求的访问范围；如不一致，则拒绝提交副作用或传递事件。
4. 在工具、元数据、任务通道、执行记录或调度路径中，分别执行相应的参数和资源检查。
5. 只有在全部检查通过后，才发布状态、将事件入队或提交资源。

3. 系统实现

本章结合当前仓库的内核与用户态源码说明各项机制的实现路径。内核代码主要位于 `os/agent_*.c`、`os/workflow_*.c` 和 `os/resource_controller.c`；用户态场景位于 `user/src/`，Nexus 还使用 `user/lib/agent_nexus.c` 和 `host_tools/agentos_relayd.py`。

实现部分	主要源文件	关键责任
启动与身份	<code>os/main.c</code> 、 <code>os/agent_identity.c</code> 、 <code>os/workflow_lifecycle.c</code>	初始化智能体子系统，分配角色、控制标识和工作流生命周期。
上下文与执行	<code>os/agent_context.c</code> 、 <code>os/agent_core.c</code> 、 <code>os/agent_execution_contract.c</code>	写入上下文，解析 V2 与 V3 请求，并按执行约定检查副作用。
任务与来源	<code>os/agent_task_channel.c</code> 、 <code>os/agent_provenance.c</code> 、 <code>os/agent_evidence_ring.c</code>	管理任务通道的 SQ 与 CQ、来源标签和工作流执行记录的阶段快照。
文件与事件	<code>os/agent_metadata_*.c</code> 、 <code>os/agent_file_state.c</code> 、 <code>os/fs.c</code> 、 <code>os/agent_live_query_events.c</code> 、 <code>os/agent_ipc.c</code>	维护元数据目录、查询与订阅、结果文件发布、消息和等待。
资源与调度	<code>os/resource_controller.c</code> 、 <code>os/workflow_credit_domain.c</code> 、 <code>os/workflow_scheduler.c</code>	处理资源使用租约、额度、工作流级选择和资源记账。
集成场景	<code>user/src/labdemo_ucore.c</code> 、 <code>user/src/agent_nexus_ucore.c</code>	驱动恢复流程和 Coordinator、System、Research 三角色常驻协作。

表 3.1: AgentOS-uCore 的实现源代码分工

3.1. Agent 进程创建与地址空间设计

3.1.1. 从普通进程到工作流中的 Agent

AgentOS 区分普通进程、协调 Agent 和专职 Agent。VFS、IPC、工具执行器和调度器读取同一份身份记录，因此同一个进程在各处拥有相同的角色和能力位。内核将 Agent 身份写入 PCB，再根据受信映像清单和父角色授权完成创建。用户态可以读取身份信息，权限判断始终以内核副本为准。普通进程仍走 uCore 原有的创建与装入路径，两类进程可以在同一系统中并存。

创建受控工作流时，内核先分配生命周期，再写入身份、角色、能力位和控制标识，随

后建立 Agent Context 页、VFS 访问范围和资源账户。任一步骤失败，内核都会回收已经分配的对象。创建完成后，各子系统都通过同一组 workflow 编号和代次找到该 Agent 所属的工作流。

用户态的 agent_create() 和 agent_create_role() 进入内核的 Agent 创建路径；agent_info() 从 PCB 返回身份、角色、配额、Loop 状态和 Context 位置，agent_workflow_lifecycle_info() 再给出完整生命周期键。创建接口负责建立对象，查询接口只复制内核已经保存的元信息，两者都不会让用户态自行填写权威身份。

agent_worker_create() 不会新建工作流根生命周期。它只在已有生命周期中创建一个携带受控待执行映像的子进程；调用者只要在同一工作流内具有 ORCHESTRATE 即可调用，不要求它必须是控制者。子进程仍须沿通常的 exec() 路径装入映像，所以该调用返回时，子进程尚未在新映像中运行。agent_scope_delegate_fd() 只接受 FD_INHERIT_DELEGATE，当前实现用于管道。该接口不会立即把文件描述符放入某个访问范围，而是在调用线程上登记一次性凭据；下一次创建受控子进程时，内核制作创建快照才会使用该凭据。子进程通过 filedup() 获得文件描述符副本，父进程继续保留原描述符，随后内核清除这张凭据。

3.1.2. 身份字段与角色策略

字段	含义	使用模块
agent_id	当前启动周期内的智能体编号	工具、上下文、观测
control_id 角色与能力位	内核分配的控制身份 角色及其具体权限集	控制者绑定、IPC 路由、句柄校验 工具、文件、消息与 artifact（工作流结果文件）操作
工作流 id+generation	工作流标识和可复用槽位的代次	生命周期、订阅、任务通道、资源、调度
VFS 访问范围	工作流的文件访问范围	元数据、查询、artifact
执行与存储账户	工作流资源归属	进程、线程、文件、块、页与缓冲区

表 3.2: 智能体身份的核心字段

内核的角色策略将每个角色映射为能力位掩码和调度权重。哨兵角色主要读取进程、元数据和调度状态；调查角色只能读取受限内容；报告角色可以创建报告结果文件；协调角色可以分派任务并创建角色。子智能体继承同一工作流的访问范围，其能力位同时受父角色授权和映像清单限制。

3.1.3. Agent Context 区的七页布局

每个 Agent 在用户地址空间的固定 ABI 地址拥有七页 Agent Context 区。前六页由内核维护，并以只读方式映射到 Guest，依次提供身份头、最近返回、上下文记录、路径索引和稳定摘要；第七页可由用户态读写，用于保存查询结果和其他高频策略数据。这样，身份、能力位、访问范围等权威状态仍受内核保护，Guest 读取 Context 和管理查询缓存时又不必为每个字段反复进入内核。

上下文最多保存 128 条记录。每条记录包含序号、请求、工具、状态、服务起始时钟、原因序号、跨度、分支代次、控制标识和记录摘要。内核按发布序号更新镜像；Guest 既可

以调用查询与快照接口，也可以直接读取同一结构。PCB 保存身份、配额、Loop 状态和路径位置等低频元信息，具体路径则通过 Context 区向用户态发布，这一划分兼顾了内核裁定与高频读取。

3.1.4. 生命周期代次与并发控制

workflow槽位可在回收后复用，内核每次重新分配时都会推进代次。订阅器、资源账户、VFS 附属记录、任务句柄和 artifact 都同时保存标识和代次。

生命周期使用普通操作、退出操作和 Workflow Fence（工作流栅栏）三类并发控制。普通系统调用增加普通操作计数；exit 或拆除过程增加退出操作计数；Workflow Fence 在两类操作全部完成后生成阶段快照。当控制者已经关闭、成员数、活跃普通操作数和退出操作数都归零，并且栅栏已经释放时，回收程序会清除该生命周期的上下文、订阅器和执行记录环。普通 VFS 访问范围会被完整回收；对于 preserve_on_retire 持久输出访问范围，只要存储账户进入 DRAINING 或 FREE，内核即可回收生命周期代次，同时保留登记项、文件和存储额度。

3.1.5. 随工作流创建的资源主体

工作流创建时，内核同时分配执行账户和存储账户。工作流中的进程、线程、文件对象、文件系统块、inode、智能体状态页和物理页都记入同一账户。这样，资源使用租约和工作流 EEVDF 都按工作流计算，子进程和多线程也始终记在原工作流名下。

3.1.6. 验证结果

agenttrust_ucore 覆盖受信 exec、角色授权、能力位收缩以及 fork、exec、exit 与普通进程共存；agentscope_ucore 覆盖访问范围与代次复用；agentfinal_ucore 覆盖上下文固定映射、快照、分支、回滚和容量淘汰。所有场景都在 RV64 uCore Guest 中通过真实系统调用运行。

3.2. Agent-OS 内核结构化交互接口

3.2.1. 工具调用协议与工具目录

Agent 既要发现可用能力，也要把工具名称、参数和返回状态交给内核可靠解析。AgentOS 因此没有让 Guest 传入任意 JSON 文本，而是定义了带版本和大小字段的工具调用协议。内核维护 26 项工具目录，每个描述项写明工具编号、名称、schema（参数模式）、标志和副作用类别；第 26 项是原生 delegate_task。启动时检查编号与名称是否唯一，并检查 schema 是否完整。

请求使用工具名称或编号加一组 typed 键值参数，响应返回状态码、Context 序号、结果类别和定长结果字段。版本不符、参数类型错误、工具不存在、能力不足和生命周期过期分别返回可区分的状态；用户态不需要从一段错误文本中猜测失败原因。目录项和请求结构都带 version 与 size，后续扩展可以在明确拒绝旧布局的前提下进行。

sys_tool_list 向 Guest 返回定长描述项。Guest 再按角色和能力位列出模型可见的工具，并补充用途、参数和结果说明。等待、上下文和模型请求等 runtime（运行时）接口

只供 Guest 使用，模型只能请求当前角色允许的产品动作。

3.2.2. 自适应的 V2 调用

V2 请求明确携带版本号、请求大小、工具名称、工具编号、参数数量和带类型信息的参数数组。当前标量类型为 uint64 与字符串，适合 uCore 的紧凑参数解析。内核先把完整参数拷入内核缓冲区，依次检查参数名、类型、数量和长度，再进入共享执行器。

V2 适合根据上一轮结果改变下一步操作的智能体循环。例如，协调智能体可以先查询失败阶段，再根据候选数决定读取摘要还是创建研究子任务。每次调用都会独立检查身份、能力位、访问范围、schema、数据来源和资源状态。

3.2.3. 四种执行路径

路径	输入形式	主要用途	运行语义
紧凑批处理	最多 64 个 agent_op	固定顺序和上下文压力场景	在一次系统调用内顺序执行，逐项返回状态
V2	带类型信息的键值参数	依据上一轮结果选择工具	每次都检查 schema、能力位和访问范围
ENFORCE V3	V2 前缀与 Execution Contract 绑定信息	预先定义的高保障 DAG	冻结节点、前驱、尝试次数、截止时间和副作用清单
任务通道的 SQ 与 CQ	16 槽 SQ 与 16 槽 CQ	有界提交、完成和背压	仅主线程建立通道，绑定提交线程并校验代次；每个已接受任务恰有一条完成 CQE，取消不会额外产生 CQE

表 3.3: 四种工具执行路径

紧凑批处理用于同步执行短操作序列。V3 则在工作流生命周期内冻结一份带版本的执行约定，其中最多包含 24 个按拓扑顺序排列的节点。每个节点关联工具、schema 摘要、所需能力位、输入和输出 artifact 类型、截止时间、尝试次数、来源与副作用清单，以及执行和存储额度。

V3 请求同时绑定执行约定代次、节点、尝试次数、来源节点、前驱上下文序号和 32 字节指纹。无论成功还是失败，完成结果都会写入固定完成槽；合法重试直接读取原结果，避免再次产生副作用。

3.2.4. 工具执行期间的资源使用租约

高风险工具可以在执行前从工作流账户预留执行和存储额度。资源使用租约依次经过 ADMITTED->ACTIVE->DEACTIVATED->SETTLED。分配器在发布对象前取得 nonce 声明，发布成功的对象继续占用同一份已用额度，析构时再结算。这样，多个对象在同一轮工具调用中同时达到资源峰值时，账户仍能准确记录用量。

3.2.5. 带类型信息的任务通道

任务通道只能由智能体主线程建立，并映射 16 槽 SQ 和 16 槽 CQ，两者各占一个 Guest 映射页；复制的请求和完成记录、权威资源状态则分别保存在两页内核私有页中。SQE 与 CQE 均为 128 字节。进入通道和资源绑定会记录提交线程标识及身份代次。内核先拷入完整 SQE，再检查进入接口的 ABI、递增请求编号、执行约定、节点、尝试次数、schema、截止时间、取消链接和带类型信息的句柄。提交者、所有者或生命周期不匹配导致 STALE 时，输出只保留 ABI 版本、结构大小和 status，其余字段全部置零；控制标识代次不匹配时，进入接口返回当前通道状态。

内核私有的头尾指针才是权威状态。CQ_FULL 只表示 CQ 暂无可写槽位，是背压，不表示已接受任务已经完成；对应任务仍待发布，直到 CQ 出现可用槽位。SQE 的环或槽位代次不一致，或者发生其他协议错误，都会使通道进入需要重新同步的状态；提交者必须以 AGENT_TASK_CHANNEL_ENTER_F_RESYNC 和空 sq_tail 显式复位，随后才能继续提交。每个已接受任务在成功、拒绝、失败、取消或超时后发布一条完成 CQE。CANCEL 命令使用独立请求编号，只引用目标任务，不会另行生成 CQE；同步策略拒绝仍会消费取消编号，并由进入接口返回 DENIED，目标任务继续保持原运行状态。目标任务已经确认或不存在时，同样会消费取消编号，进入接口返回当前代次和水位线。Task Bridge 既接受空输入，用于验证通道、截止时间、取消和完成语义；也允许 ECHO Provider（服务提供者）读取 UTF-8 资源 snapshot，检查带类型信息的输入能否进入同一个工具执行器。

任务输入也可以由 IMPORT 控制请求从当前进程的只读普通文件描述符导入。请求进入时，内核先固定文件引用和当次 workflow/contract cut；实际读取通过带租约的 readi 从 offset 0 多探测一个字节。只有文件恰好在给出的 1-63 字节处结束、内容不含 NUL 且 UTF-8 合法时，才建立资源。导入提交前，内核在同一条事务 lane 中再次核对 Context 序号、哈希和来源标签，避免把文件内容与另一时刻的 Context 拼成一条来源记录。

导入过程不改变文件描述符的共享 offset，也不把 structfile* 留在通道中；内核只在私有资源槽保存不可变的 inline snapshot、SHA-256、最新 Context 序号和 UNTRUSTED_FILE_DATA 标签。IMPORT 只登记输入，不另行追加 Context 记录。

资源句柄由 slot、type 和 generation 共同标识。BORROWED 输入执行后仍保持 LIVE，可由后续任务继续引用；OWNED 输入在完成后自动消费，显式 RELEASE 也会使旧句柄变为 STALE。ECHO Provider 从内核 snapshot 复制 payload，不再依赖导入文件的后续内容或 offset。若 IMPORT 已成功而最终结果无法复制回用户态，内核会撤销尚未暴露的资源，避免只有内核知道的槽位泄漏。

3.2.6. 原生 delegated Task Channel

跨 Agent child Task 不再封装为用户态消息状态机。Coordinator 把准确的 56 字节 AGENT_ARTIFACT_TASK 描述符导入 Task Channel；描述符绑定目标 PID、Agent 编号、控制标识、任务关联和 Guest capsule handle，大段输入与结果仍由 artifact 承载。delegate_task SQE 进入 Execution Contract 和内核 pending 队列后，只有持有 AGENT_CAP_TASK_ACCEPT 且取得独立 AGENT_IPC_ROUTE_TASK 路由的目标 Agent 才能领取。

目标通过系统调用 agent_task_delegate_claim（编号 567）取得不可变描述符，完成业务与结果 artifact 后，再通过 agent_task_delegate_complete（编号 568）提交终态。claim 建立的 delegated effect lease 精确绑定执行者 PID、Agent、control_id、线程 identity

generation 以及 channel/request/slot 代次；每次直接作用必须是 Execution Contract 副作用清单的子集。正常完成、协作式终态确认或目标进程 quiescence 后，内核才在活动引用归零处撤销该租约。

同一活动生命周期内的 controller 也可复用 complete 接口，以 AGENT_TASK_DELEGATE_COMPLETE_F_REQUEST_CANCEL、CANCELLED、零 ACK 字段和准确的 owner/channel/request/slot/task/correlation tuple 请求取消。内核除核验该完整绑定外，还要求调用者同时持有 AGENT_CAP_ORCHESTRATE、AGENT_CAP_WAIT_CANCEL，并具有指向 owner 的活动 AGENT_IPC_ROUTE_TASK 路由。返回 OK 表示取消请求已被接收，任务终态仍以 terminal CQE 为准。

若该取消、截止时间、owner 退出或生命周期关闭在 claim 后先取得终态优先级，complete 返回带 generation 的终态 offer。处于 CLAIMED 的 provider 须先清理预绑定输出，再以 ACK_TERMINAL 和准确 offer 确认；内核随后只向 owner 发布一条 terminal CQE。首版协议采用协作式终止，无法强制终止永久无响应的 provider。一般 MESSAGE 路由仍用于进程通信，child Task 的正文、状态和结果由 Task Channel 与 artifact 承载。

3.2.7. 验证结果

agenttoolabi_ucore 覆盖 26 项目录发现、名称与编号一致性、未知参数、类型错误和返回缓冲区；agentcontract_ucore 覆盖 DAG、截止时间、重试、取消、来源 Context 与来源检查；agenttask_ucore 除了 SQ/CQ 已满、重新同步、取消和完成记录保留，还检查 UTF-8 与 TASK snapshot、BORROWED/OWNED 生命周期、generation 防 ABA、TASK route、claim/complete、controller 取消请求、终态 offer 确认和唯一 CQE。最终 copyout 失败后的回滚分支由结构检查、mutation 测试和资源表模型核对。任务通道的测量数据另保存了 96 条每条 16 个操作的序列，第 4 章集中分析其分布。

3.3. 上下文路径管理、数据来源与 Workflow Fence

3.3.1. 从多轮调用到 Context Path

Agent 的下一步决策往往取决于前几轮工具结果。上下文路径管理把这些请求和结果按前驱关系组织为 Context Path：每次工具调用即将执行可能产生副作用的操作时，AgentOS 建立一条上下文记录，并由附属记录保存归一化的 agent_op、结果和 provenance（来源追溯）信息。附属记录不照搬完整的 V2 参数、V3 绑定或任务通道 SQE，而是保留后续决策真正需要的请求、结果、因果关系和数据来源。

记录序号只在同一次清空周期内递增。记录包含请求、工具、状态、服务起始时钟、原因序号、跨度、分支代次、控制标识、短载荷或结果以及记录哈希；执行清空后，序号从 1 重新开始。path_parent_sequence 指向当前活动头记录，后继索引用于查询分支。

上下文追加依次经过准备、填充、发布和提交；detail 会在记录标为奇数前写入。镜像头发布记录数、头记录、最早记录、最新记录、可见头记录、丢弃数和回滚数。Guest 读取记录后重新核验序号和哈希，并再次确认摘要未变化，便可得到一致快照。

用户态通过 context_push() 追加记录，也可以用 context_query() 复制活动路径；高频读取时则直接检查 Context 镜像。context_rollback() 把可见头移到仍在窗口内的历史记录，context_clear() 清空当前路径。系统调用与直接映射读取使用相同的序号

和摘要，因此两种读取方式可以互相核对。

3.3.2. 分支、回滚与有界淘汰

上下文路径最多保留 128 条记录。容量用尽后，内核按序号淘汰最早记录，并更新最早记录、最新记录和丢弃计数。活动路径只引用仍然保留的前驱记录。

回滚将 `visible_head` 移到仍然存在的上下文序号，并为后续调用分配新的分支代次。已经产生的文件、IPC 和模型服务副作用保持不变。需要幂等的副作用由 V3 尝试缓存和具体工具协议处理。因此，上下文分支只改变后续决策看到的路径；手工备注、回滚和清空都不会预留或提交执行记录。

3.3.3. 数据来源传播

AgentOS 使用六类可组合的数据来源标签，标签随上下文、文件、工具返回、IPC 和 artifact 传播。

标签	含义	典型来源
KERNEL_FACT	内核生成的结构化事实	PID、资源、调度器、能力位
TRUSTED_USER_CONTR OL	经产品界面提交的用户控制信息	新交互轮次、参数绑定 审批
AGENT_DERIVED	智能体根据已有输入推导的数据	摘要、计划、报告
UNTRUSTED_FILE_DAT A	来自文件的内容或元数据	查询、读取、摘要哈希
UNTRUSTED_TOOL_OUT PUT	工具执行返回	带类型信息的 V2 或 V3 结果
CROSS_AGENT_DATA	经 IPC 或 artifact 在角色间传递	任务结果、协调者整理的 artifact

表 3.4: 上下文的数据来源标签

文件、工具、邮箱和任务资源按保守的按位或规则传播来源标签。生成摘要、重新计算哈希和在智能体之间整理 artifact 时会增加新来源，同时保留既有标签。文件数据保留 UNTRUSTED_FILE_DATA 来源，控制授权由能力位和 Execution Contract 独立判定。

3.3.4. 工具清单与副作用检查

每个内核工具都有安全清单，其中列出可接受的输入来源标签、输出新增标签、所需能力位，以及文件、元数据、IPC、进程、权限、artifact 和订阅的副作用掩码。启用 ENFORCE V3 后，生命周期代次、执行约定中冻结的边、schema、能力位、工具清单和数据来源必须同时匹配。失败请求会在工具产生副作用前返回结构化状态。只有成功预留并提交关键记录的拒绝才取得执行记录票号；空间不足时调用返回 NO_SPACE 或 RETRY。

3.3.5. 执行记录环

每个活动 workflow 按需分配四个智能体状态页：三页普通记录环，共 48 个 256 字节槽位；一页关键记录环，共 16 个槽位。直接调用或 V3 调用的标准完成或拒绝路径会预留递

增的执行记录票号，填充后再提交；手工备注、回滚和清空不使用这套预留和提交规则。当较早编号仍在准备时，读取者会停在当前位置，从而保持全局发布顺序。

普通成功的上下文记录写入普通记录环，获准副作用和成功建立记录的拒绝写入关键记录环。拒绝记录无法预留或提交时，直接调用或 V3 调用返回 NO_SPACE 或 RETRY，关键记录票号保持为空。复用环形空间前，已提交分段会汇入内部根记录。该结构为 Workflow Fence 提供按执行记录票号排序的执行记录。

3.3.6. Workflow Fence

控制者通过 `agent_run(count=0, AGENT_RUN_F_FENCE)` 发起 Workflow Fence。内核检查协调者与控制标识的关系，使生命周期进入屏障状态，排空元数据、实时查询延迟工作和 VFS 回收，检查执行与存储账户中的待结算额度归零，最后密封执行记录环。

屏障请求携带版本号、`struct_size`、必须为零的 `flags` 与 `reserved`、挑战值和请求编号；前序根记录由内核在阶段快照完成后写入返回回执。请求和回执可以同时为 NULL，此时内核执行匿名且不等待结果的屏障。提供回执时，生成的 320 字节回执绑定前序根记录、挑战值、屏障序号、执行记录票号范围、事件与缺口计数、元数据代次、额度周期、精确已用额度摘要和有序分段摘要。请求编号非零且提供回执时，内核在内部状态提交后复制结果；同一个请求编号再次复制时返回缓存结果。该回执表示当前启动周期内的工作流阶段快照，其标志同时记录元数据是否易失以及覆盖状态。

3.3.7. 验证结果

`agentfinal_ucore` 连续执行 64 次工具调用，并覆盖快照、分支、回滚和窗口淘汰。数据来源场景把文件来源传入受保护的副作用，验证副作用前的拒绝与关键记录环记录。屏障场景覆盖挑战值、重试缓存、待结算额度、元数据代次和结果复制重试。

3.4. 面向 Agent 查询优化的文件系统扩展

3.4.1. 从路径访问到属性查询

科研和恢复 workflows 经常需要回答两个问题：当前哪些对象满足条件，哪些对象刚刚发生变化。传统路径访问适合已知文件名的场景，却不能直接表达“查找某个阶段、状态或类别的结果文件”。AgentOS 在 uCore VFS 之上增加按启动周期隔离的元数据目录，为显式登记的文件维护项目、运行、阶段、状态、类别、依赖关系和内容摘要，使 Agent 可以按属性与摘要查询对象，也可以订阅后续变化。

3.4.2. 元数据与 inode 代次

拥有 META_WRITE 的智能体调用 `agent_file_meta_set()` 写入元数据，字段与真实 `dev+inum+incarnation` 绑定。更新通过 `update_mask` 完成，删除使用显式 DELETE 动作。文件创建和重命名仍沿用 uCore VFS，只有工作流明确关心的对象才会进入元数据目录。

对象代次用于区分 inode 编号复用前后的对象。VFS 以增量方式维护它，并随查询和订阅结果返回。查询或订阅命中记录后，内核会复核元数据目录代次、生命周期、访问范

围和完整条件。unlink、内容大小变化和对象失效都会更新已登记的附属记录。

3.4.3. 遍历查询与选择性索引

元数据目录为 status、stage 和 kind 维护选择性索引。带类型信息的查询只携带条件字段、查询标志和最大命中数；访问范围由调用者身份限定，查询计划由内核选择。结果返回命中数、候选记录数、扫描记录数、全部命中数、实际返回数、truncated、查询计划、计划原因和文件系统代次。调用者可以用 SCAN 强制遍历，也可以用 USE_INDEX 允许内核使用索引。启用索引时，内核按固定的 status->stage->kind 顺序选择第一个已给定字段，不比较字段基数；索引路径仍会对候选记录检查完整条件，因此遍历与索引查询返回相同结果。最大返回数为 8，total_hits 仍报告全部命中数；当前 ABI 没有分页游标。

查询结果是带属性、摘要、digest 和对象代次的定长结构。Guest 可以把输出地址直接放在 Agent Context 区的第七页用户缓存中。用户缓存页由内核映射并计入配额，查询结果的保留策略由 Guest 管理。

3.4.4. 带类型信息的实时订阅与缺口重新同步

agent_live_watch() 安装一份完整的 agent_file_query。订阅器只保存查询条件、访问范围和代次，不保存持久结果集。每次元数据变化时，内核重新计算变更前后是否满足条件：对象从未命中变为命中时产生 ENTER，对象持续命中且记录改变时产生 UPDATE，对象从命中变为未命中或被删除时产生 LEAVE。事件以 AGENT_EVENT_FILE_QUERY 写入目标智能体的上下文和有界队列。

队列或待处理表无法保存全部增量时，内核会推进 resync_generation 并发送 RESYNC_REQUIRED。连续的文件系统代次变化发布 ENTER、UPDATE、LEAVE；出现缺口后才进入重新同步。

恢复时，智能体保留旧订阅，先以相同条件安装不带 ACK_RESYNC 的替换订阅，再执行有界查询。完整基线要求快照满足 truncated=0；随后在旧订阅上以对应代次执行 ACK_RESYNC 并取消旧订阅，替换订阅覆盖整个切换时段。单次查询最多返回 8 个命中项，当前 ABI 没有分页游标，快照截断时调用者需要收紧查询条件。

Workflow Fence 会排空删除标记、内容增量并尝试投递标记事件。工作流或访问范围中的重新同步尚未由 ACK_RESYNC 确认时，屏障返回 RETRY；最终回收阶段再清理残留对象。

3.4.5. 查询性能

实验使用目录规模 24、64、96 与精确命中数 1、2、4、8，组成 12 个实测网格。每个网格保留 15 个 AB/BA 内部配对，并记录遍历和 indexed 查询的延迟、候选数和检查记录数。

图3.1给出的 12 个格点中，indexed 路径的中位加速比为 1.164–2.808 倍；目录规模从 24 增至 64、96 时，indexed 更快的内部配对数分别为 43/60、60/60、58/60，整体收益随规模增大而上升。status->stage->kind 的固定选择顺序先缩小候选集，再逐项复核完整条件。命中数与加速比并不单调，24 项目录中仍有 17/60 个配对不占优；这些样本使用 ready index，未计入建索引和维护索引的成本。收益主要出现在目录较大、筛选条件明确的已登

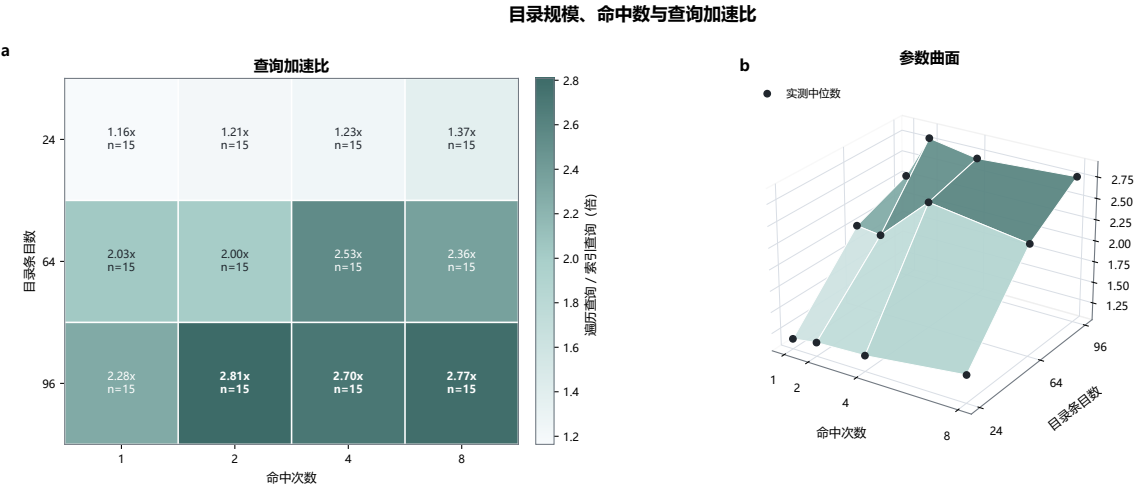


图 3.1: 目录规模、命中数与 indexed 查询加速比

记对象上，后续测量还将纳入索引维护开销和低选择性查询。

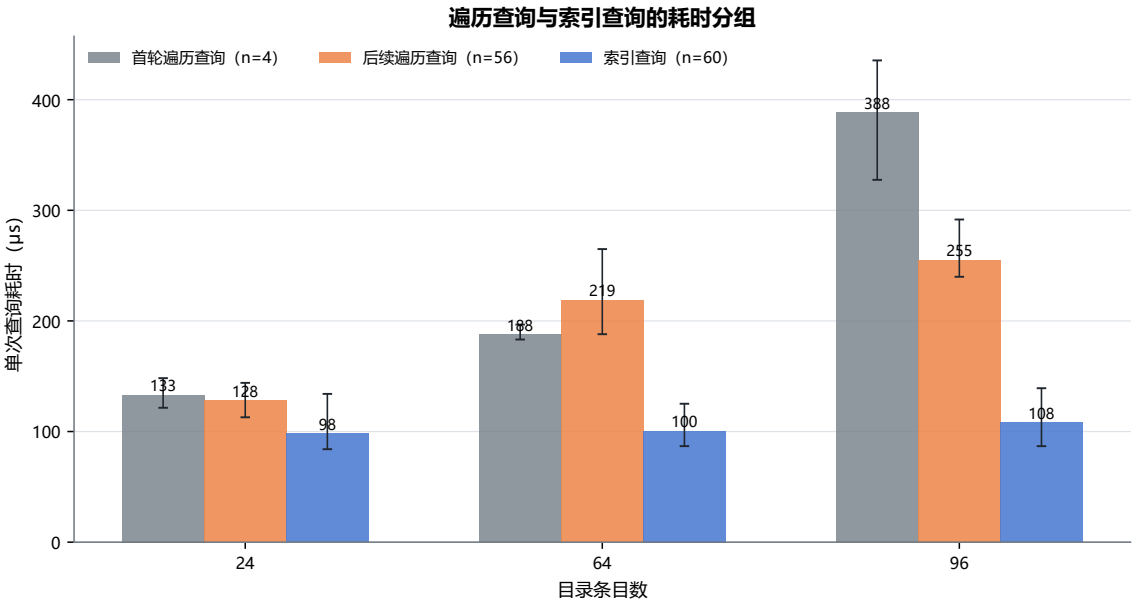


图 3.2: SCAN 与 indexed 查询的耗时分组

图3.2显示，indexed 查询的中位耗时随目录规模由 98.3 微秒升至 108.3 微秒；首次 SCAN 为 388.5 微秒，后续 SCAN 为 255.1 微秒。28 条诊断记录均为 `cache=ready`，因此首次 SCAN 对应该阶段的首次完整遍历。固定索引选择减少了中位延迟及其对目录规模的敏感性，但 64 项目录的后续 SCAN 仍出现 25.7505 ms/query 的尖峰，说明尾延迟尚未稳定。当前实现将 indexed 查询作为有明确筛选字段时的默认路径，同时保留 SCAN 作为语义等价的回退，并继续定位完整遍历和目录检查造成的长尾。

3.4.6. 验证结果

`agentfs_ucore` 覆盖元数据设置、更新、删除、摘要、摘要哈希、对象代次和访问范围；`agentbench_ucore` 使用同一数据集比较遍历与索引查询的结果集、候选数和延迟。实时查询场景覆盖 ENTER、UPDATE、LEAVE、队列缺口、RESYNC_REQUIRED 以及替换订阅、快照、确认的安全切换。

3.5. Agent Loop 内核运行机制

3.5.1. 从轮询循环到内核等待

Agent Loop 会等待用户输入、模型回复、文件变化和其他 Agent 的消息。持续轮询不仅频繁进入内核，还会让大量等待线程占用调度时间。AgentOS 使用有界事件队列、订阅和等待接口，让没有事件的线程真正睡眠；消息、文件事件或 TIMER 到达后，内核再把相应线程唤醒。

每个智能体拥有容量为 16 的事件队列，其中保留一部分内核槽位，同一事件来源最多排队 4 条事件。路由表同样具有固定容量。发送 MESSAGE 时，内核检查 MESSAGE_SEND 能力位、活动工作流、发送者、目标路由和订阅状态。公开的 agent_event 只返回发送者、目标 PID、关联号等事件字段；control_id 和数据来源标签保存在内核附属记录中。

Agent 使用 agent_watch() 登记关心的事件，以 agent_wait() 休眠等待，再用 agent_unwatch() 取消订阅。心跳由 agent_heartbeat_set() 和 agent_heartbeat_stop() 调整。Loop 的下一步决策仍在 Guest 中完成，内核只负责保存订阅、管理等待状态，并在事件或 TIMER 到达时提供可靠唤醒。

非 TIMEOUT 的 agent_wait 会先检查已排队事件。选中事件时，内核建立保留项；没有事件时，内核以线程身份代次作为等待键安装等待者并让线程睡眠。事件入队和等待者状态使用相同锁序，确保到达事件与等待代次对应。TIMEOUT 路径在本地直接返回，不建立保留项，也不写入上下文。agent_wait_cancel() 仅向等待路径注入 AGENT_EVENT_CANCELLED，工具和任务取消由各自协议处理；消费事件后才更新循环状态和上下文。

线程进入工具执行、等待事件和结束本轮处理时，内核分别维护 RUNNING、WAITING 和 IDLE 状态，并把同一进程中各线程的状态汇总到 PCB。agent_info() 可读取这一状态，观察程序也会把它与唤醒和调度记录对应起来。任务结束后，Guest 沿普通 exit 路径退出；生命周期在成员、在途操作和资源引用排空后统一回收 Context、订阅与队列对象。

3.5.2. 心跳与模型请求关联号

Agent 可以设置心跳间隔。定时器到期时，内核生成 TIMER 事件并唤醒 Agent；Guest 再根据当前状态决定检查文件、发送消息或继续等待。停止心跳会阻止后续 TIMER 产生，但已经进入事件队列的 TIMER 仍按普通事件消费，避免悄然改写队列历史。

LLM_REQUEST 与 LLM_RESPONSE 复用消息与等待机制。请求会登记生命周期、请求者和中继程序的 PID、控制标识、关联号和截止时间。同一请求者的关联号严格递增，并且只有成功入队后才更新。中继程序返回匹配响应时，内核原子消费待处理请求并生成 LLM_DONE。只有同一关联号仍处于活动待处理状态时才返回 DUPLICATE；复用已经完成或过期的关联号则返回 CONFLICT。待处理请求使用 120 秒内核时钟 TTL，完成历史区分已消费请求和超时请求。

3.5.3. 工作流资源账户

资源账户维护已用额度 U 、待结算额度 P 、空余额度 F 和总额 $U + P + F$ 。当前账户的可用额度先在本地图态中转移：

reserve:F→P commit:P→U cancel:P→F release:U→F

账户补充额度时会检查全局限制、资源类别和账户硬限制。系统压力、上下文切换、账户关闭和 Workflow Fence 都会收缩 F 。Workflow Fence 在同一把锁内执行 trim+snapshot，要求 $P = 0$ ，并导出精确 U 、账户代次、额度周期和向量摘要。

3.5.4. 按 workflow 汇总的 EEVDF

传统线程级公平会让多线程 workflow 得到更多调度候选位。AgentOS 将同一资源账户中的线程合并为一个 workflow 实体。调度器使用 workflow 虚拟运行时间 (vruntime) 计算滞后量，满足 $vruntime \leq global_vtime$ 的实体进入可服务集合，再从中选择虚拟截止时间最早的实体。延迟类别、事件和截止时间可以缩短服务请求。一次分派完成后，内核根据实际服务周期更新 workflow 虚拟运行时间；睡眠实体的滞后量会向全局虚拟时间衰减。

只有一个 workflow 实体时，调度器使用快速路径。实体表容量为 4，其中包含一个 `BOOT_SEALED` 启动参与者，因此最多可同时纳入三个新建 workflow。实体信息不完整或容量超出时，调度器回到原 uCore 路径，并增加回退计数。

3.5.5. 唤醒延迟与公平性

唤醒与公平性测量使用 6 次全新 QEMU 启动，覆盖并发度为 1、2、3、4 的服务窗口和精确唤醒探针。504 条探针直接读取调度器跟踪中的 `ready_age`，其中 425 条为 0 个时钟节拍 (tick)，79 条为 1 个 tick。uCore 的 tick 为 10 ms，这些记录表明等待者均在一个时钟节拍内得到服务。

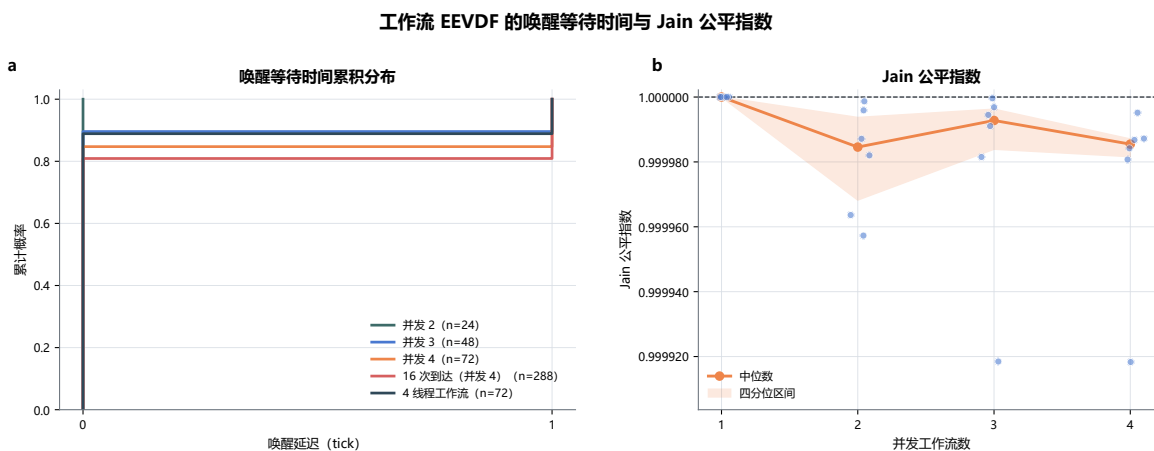


图 3.3: 工作流 EEVDF 的唤醒 tick 与调度公平性

图 3.3 中，并发度为 1、2、3、4 时，Jain 指数中位数分别为 1.000000、0.999985、0.999993 和 0.999985。工作流 EEVDF 将同一 workflow 内的线程折叠为一个调度实体，抑制了线程数放大带来的服务倾斜。现有结果来自单 Hart、10 ms tick 和最多四个实体的负载；在独立的四线程放大场景中，Jain 中位数为 0.9980725，该 workflow 取得 26.895% 的服务份额。后续将在更细粒度计时、更多实体和 SMP 环境中复核。

3.5.6. 验证结果

`agentloop_ucore` 覆盖原子等待、唤醒、超时、取消、心跳和消息路由；`agentsched_ucore` 与 `agent_eevdf_ucore` 覆盖快速路径、实体容量、线程放大、睡眠衰减、回退、唤醒直方图和 Jain 公平性。

3.6. AgentOS 控制台综合工作流

3.6.1. 科研恢复场景

AgentOS 控制台用六组内核机制执行一个确定性的科研恢复工作流。labdemo_ucore 创建协调、哨兵、调查和恢复四个角色，建立工作流身份、MESSAGE 路由、上下文、元数据和资源账户。场景登记 prepare、align、analyze、report、archive 五个阶段及其依赖关系，再查询失败节点、读取摘要、向其他智能体发送恢复请求，完成 align 恢复和 report 更新。

固定策略为每个阶段指定相同的业务工作量，工具结果仍由 AgentOS 内核执行和检查。因此，该场景同时覆盖身份、上下文、实时查询、IPC 等待和恢复写入，并能在不同实现路径之间进行等量比较。

3.6.2. 遍历与索引路径的配对实验

综合场景保留遍历实现和索引实现，两者功能等价。每次独立 QEMU 启动按 A/B 或 B/A 顺序运行，两侧产生相同的工作负载和结果指纹。核心窗口覆盖查询、恢复写入、fsync 和复核，查询区段单独记录纯查询时间。

测量数据来自 16 次独立工作流启动，AB 与 BA 各占一半。16 个核心查询区间中，索引路径的延迟均更低；遍历和索引路径的核心中位数分别为 34.713 ms 与 13.294 ms，配对加速比中位数为 3.118 倍。

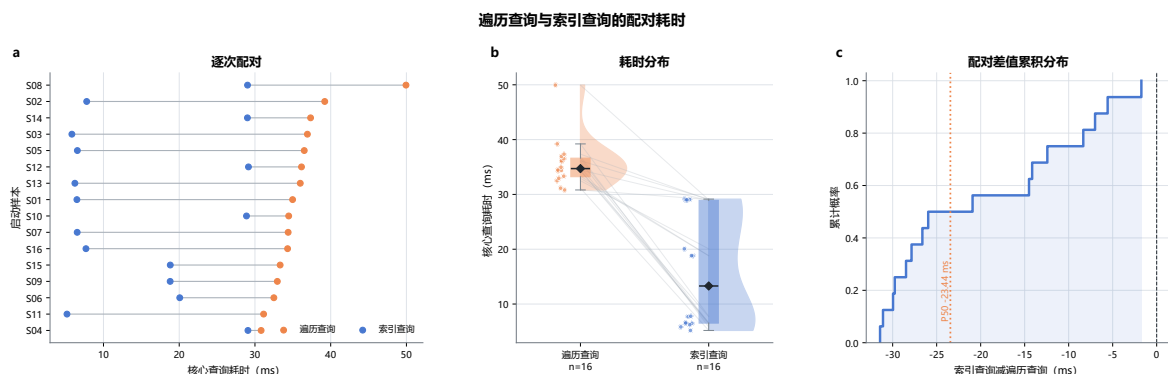


图 3.4: 遍历查询与索引查询的配对耗时

图3.4显示 16/16 个核心查询配对均由 indexed 路径更快完成，中位加速比为 3.118 倍。选择性索引先减少候选记录，再执行与遍历路径相同的完整条件复核，收益来自减少无关扫描。AB 与 BA 的中位加速比分别为 1.454 倍和 5.481 倍，indexed 路径的四分位区间为 6.5185–28.9128 ms，表明收益幅度受启动顺序和运行状态影响。下节进一步比较核心查询与完整流程。

3.6.3. 核心查询与完整流程窗口

核心窗口用于观察 AgentOS 查询路径，完整流程窗口在此基础上还包含场景启动、文件与元数据准备、工作流收尾和统计输出。在完整流程窗口中，索引路径只在 16 个配对中的 3 个取得更低延迟，indexed-traversal 的中位数为 +13.452 ms。这说明核心窗口以外的总开销已经抵消索引收益；当前计时没有把这些外围步骤逐段拆开，尚不能确定其

中哪一步占主导。

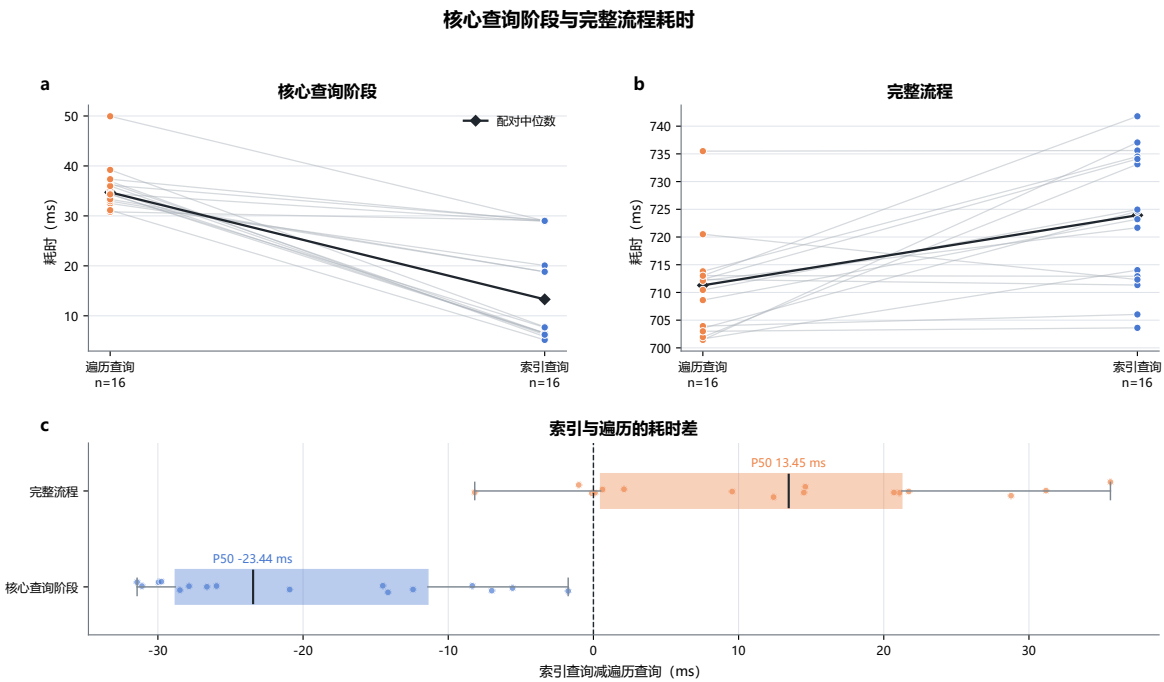


图 3.5: 核心查询阶段与完整流程耗时

图3.5把核心查询段与完整流程分开后，核心段的 `indexed-traversal` 中位差为 -23.4415 ms，16/16 个配对更快；端到端中位差却为 $+13.452$ ms，只有 3/16 个配对更快，整体约慢 1.90%。`indexed` 路径在核心查询以外的余量在 16/16 个配对中更长，中位多 33.477 ms，说明完整流程的准备、统计和清理抵消了核心查询的改进。后续将分别计时这些步骤，定位主要开销。

3.6.4. 系统结果

综合工作流依次执行智能体创建、上下文记录、带类型信息的工具调用、实时查询、消息等待、恢复写入和执行记录。输出包含业务指纹、各阶段状态、检查记录数、核心与完整流程时间以及工作量计数。遍历查询和索引查询都在同一 AgentOS-uCore 内核与同一 Guest 场景中运行，并以相同的用户态约定比较两种查询实现。

3.7. 综合场景：AgentOS Nexus 多智能体协作平台

3.7.1. 长驻会话与三角色协作

AgentOS Nexus 是构建在 AgentOS-uCore 之上的通用交互 Harness。它接受任意非空用户任务，模型可以直接回答，也可以自主选择 `search_files`、`read_file` 或 `inspect_system`。一轮会话在同一工作流中常驻 Coordinator、System 和 Research 三个业务 Agent；每个角色都有独立的 PID、Agent 编号、控制标识、能力位和 Context，并共享工作流生命周期、VFS 访问范围、资源账户和 EEVDF 实体。

产品角色/内核角色	职责	主要权限
Coordinator / 协调者	维护会话、调用模型、创建 root/child Task、核验返回	建立 Task Channel、授予 TASK route、读取结果 artifact
System / 哨兵	读取当前 Guest 的状态、进程或 Context	领取系统观察任务、执行只读工具、发布结果 artifact
Research / 调查者	使用 Guest Catalog 选择的工作区对象研究源码与文档	领取研究任务、读取版本绑定字节、发布结果 artifact

表 3.5: Nexus 的三个业务角色

Guest Relay 负责 QEMU 串口帧和模型循环。Host Relay 持有本地套接字、传输加密配置、服务接口密钥和模型格式适配器；Host workspace broker 则只处理显式会话 root 内的 manifest 与字节传输。任务所有权、工作区候选、结果 artifact、Context 和任务终态都保留在 Guest 与 AgentOS 内核中。

3.7.2. 一次交互回合

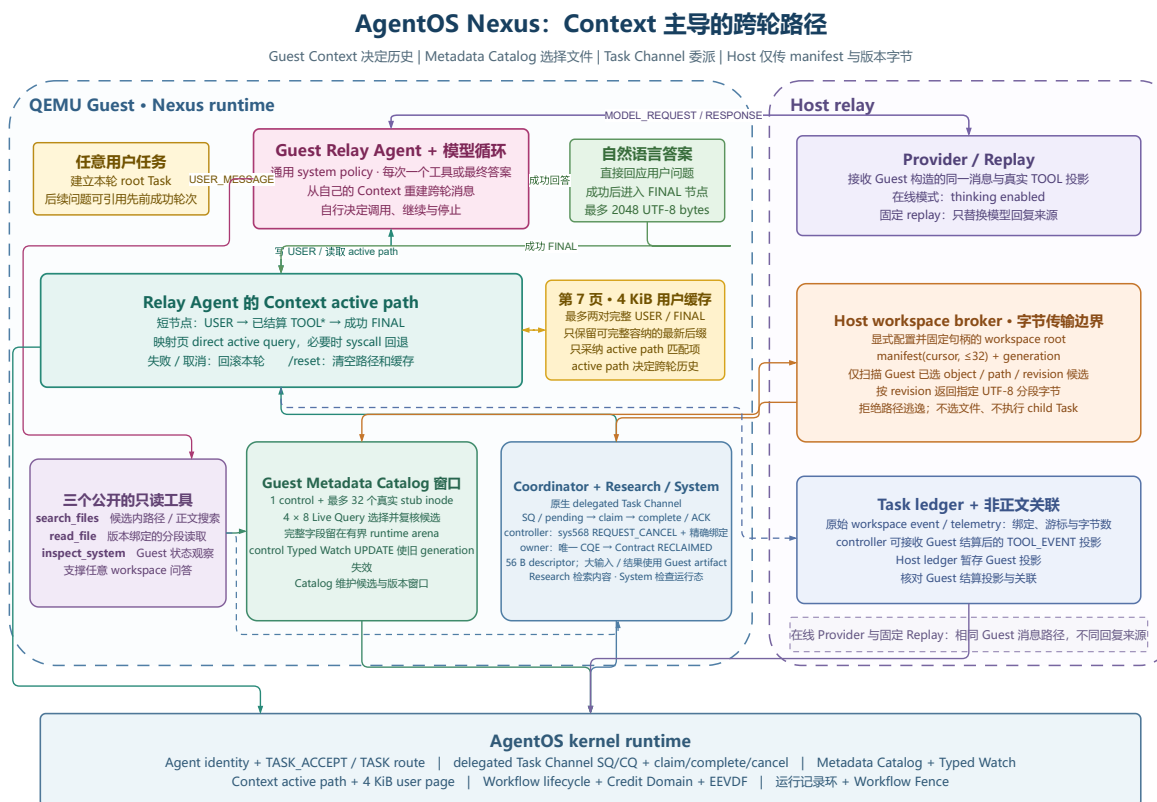


图 3.6: Nexus 中由 Context、Catalog 与 Task Channel 共同驱动的协作流程

1. Guest 把用户输入建立为本轮 root Task, 并将 USER 短节点写入 Relay Agent 的 Context active path。
2. Relay 直接查询 active path, 并用第七页用户缓存补充仍然可见的有界正文, 形成下一次 MODEL_REQUEST。

3. 模型可以直接返回 FINAL，也可以选择一个公开工具。inspect_system 交给 System；search_files 和 read_file 交给 Research。
4. Coordinator 把目标身份、任务关联和 capsule handle 写入 56 字节描述符，经 Execution Contract 和原生 Task Channel 提交 child Task。
5. 目标 Agent claim 任务，读取 Guest capsule artifact，执行受能力位与副作用清单约束的操作，再发布结果 artifact 并 complete。
6. Coordinator 从唯一 terminal CQE 结算，核验结果 artifact 后把真实 TOOL 结果写入 Context，并交给下一轮模型决策。
7. 成功 FINAL 采用准备与提交两阶段写入 Context；失败或取消则回滚本轮路径，避免未结算内容进入下一轮历史。

3.7.3. 原生 delegated Task Channel

Nexus 不再用用户态消息状态机模拟 child Task。内核工具目录中的第 26 项 delegate_task 接收准确的 56 字节 AGENT_ARTIFACT_TASK 描述符；描述符只绑定目标 PID、Agent/control 身份、任务和父任务关联、capsule handle 与保留字段，大段任务正文和结果留在 Guest artifact 中。

Coordinator 是 Task Channel owner，也是单一 issuer。目标必须持有任务接收能力位，并获得与普通消息分离的 TASK route。提交成功后任务进入 Execution Contract 的 RUNNING 状态和内核 pending 队列；System 或 Research 使用 claim/complete 系统调用领取并提交终态，owner 只收到一条 terminal CQE。相关 ABI 名称和系统调用号如下：

```
1 capability AGENT_CAP_TASK_ACCEPT
2 route      AGENT_IPC_ROUTE_TASK
3 syscall 567 agent_task_delegate_claim
4 syscall 568 agent_task_delegate_complete
```

claim 同时建立 delegated effect lease，精确绑定 worker 身份、线程 generation 与 channel/request/slot 代次。worker 或已登记 helper 的直接副作用必须是 Contract manifest 的子集，complete 以及终态确认都要等待活动 lease 引用归零。

同一活动生命周期内的 Nexus controller 通过上述 complete 接口设置 AGENT_TASK_DELEGATE_COMPLETE_F_REQUEST_CANCEL 发起取消：它提交 CANCELLED、零 ACK 字段以及准确的 owner/channel/request/slot/task/correlation tuple；内核还要求 AGENT_CAP_ORCHESTRATE、AGENT_CAP_WAIT_CANCEL 和指向 owner 的活动 AGENT_IPC_ROUTE_TASK 路由。返回 OK 表示取消请求已被接收，任务终态仍以 terminal CQE 为准。若任务已经处于 CLAIMED，provider 须清理预绑定输出，再以 ACK_TERMINAL 确认最新 generation 的 offer。首版采用协作式终止，无法强制终止永久无响应的已领取任务。

3.7.4. Guest Catalog 与 Host 工作区边界

Host workspace broker 在会话开始时固定一个显式 root，并以 generation、object id、相对 path、revision、大小和类型构造分页 manifest。revision 来自与后续搜索或读取相同的已验证文件句柄和字节；broker 拒绝绝对路径、目录跳转、符号链接逃逸和超出边界的对象。候选对象由 Guest Catalog 选择。

Guest 为当前 manifest 页面建立一个 control inode 和最多 32 个 data-stub inode，把对象登记到 AgentOS Metadata Catalog。每页按四个 stage 执行最多 8 项的 Live Query，并在有界 runtime arena 中复核完整 path、object id 与 revision。control inode 上的 Typed Watch 接收 generation UPDATE；如果出现 RESYNC_REQUIRED 或 stale 结果，Guest 使旧窗口失效，从 cursor 0 重建 Catalog 与订阅。

search_files 先由 Guest Catalog 选出候选，Host 在这些 object/path/revision 绑定的候选中执行正文匹配。read_file 先用完整路径在 manifest 中定位对象，再由 Catalog 精确复核，Host 返回该 revision 的指定行段。Metadata Catalog 负责有界对象登记与候选选择，正文匹配由 Host 在 Guest 指定的候选集合内完成。路径、manifest 游标、字节预算与输出大小均有明确上限。

3.7.5. artifact、来源与 Context

Host 返回的实际搜索或读取正文首先进入 Guest，成为 Research 的输入 artifact。Research 核对任务、对象绑定、摘要与来源后，以自身身份发布结果 artifact；manifest 同时记录生命周期、句柄 generation、任务/父任务、producer、owner、权限、载荷大小、provenance label 和 producer Context sequence。Coordinator 只有在 Task Channel terminal CQE 与 artifact 彼此匹配后才读取结果。

Coordinator 把已结算的工作区结果写为 TOOL Context，Relay 再从 Guest Context active path 重建下一轮消息。公开给模型的投影可截断或结构化，其来源仍绑定同一个 Guest artifact；inspect_system 的观察结果以不可信运行数据进入 Context，控制授权由能力位和 Execution Contract 判定。

3.7.6. 跨轮 Context 路径

Nexus 的跨轮历史由 Guest Relay Agent 的 AgentOS Context active path 决定。每轮 USER、已经结算的 TOOL 和成功 FINAL 都形成短 Context 节点；第七页 4 KiB 用户缓存只为仍在 active path 上的 USER/FINAL 对补充有界正文。Relay 优先直接读取 Context 映射页，必要时回退到系统调用查询。

失败或取消会把 active head 回滚到本轮 USER 之前，/reset 同时清空 Context、用户缓存和工作区 Catalog/watch 状态。在线 Provider 与固定 Replay 都接收由 Guest Context active path 重建的消息和 TOOL artifact 投影，Nexus 的跨轮历史由同一条 Context 路径维护。

3.7.7. 工具发现与内核观测

Nexus 启动时读取 AgentOS 内核的 26 项 Tool Registry，检查名称、编号和 schema，再按业务角色、内核角色、能力位和副作用类别筛选。模型公开表面保持三个通用只读工具：search_files、read_file 和 inspect_system；delegate_task 是 Harness 在 Guest 内部使用的内核工具，不要求模型构造 Task descriptor。

观察路径通过 timeline、audit、identity、Context、Task Channel 和 Catalog 状态生成结构化事件。它可以显示 TASK route、claim/complete、terminal CQE、artifact 接收、Context sequence、Catalog generation、Typed Watch 失效以及 Agent 的等待和调度状态。任务状态机由 Guest 与内核维护，controller 接收已结算投影和非正文关联信息。

3.7.8. 运行验证

固定 ReplayProvider 的每条响应都与 Guest 当次生成的规范 MODEL_REQUEST SHA-256 绑定，并在当前 RV64 镜像的真实 QEMU 启动中执行。回放与在线模式使用同一个 Guest 模型循环、Context active path、Task Channel、Metadata Catalog、Live Query、Typed Watch 和 artifact 路径；固定数据只替换 Provider 回复，不替换工具执行结果。回归覆盖直接回答、模型自主选工具、版本化工作区读取、跨 Agent claim/complete、controller 取消请求、provider 终态确认、Context 回滚/清空与正常会话关闭。

3.8. 实现演进与核心片段

本节沿七条演进线说明系统如何从初版方案收敛到当前实现。每条演进线先呈现原有问题与结构变化，再用前后对照图、核心代码片段和回归结果说明最终机制。

3.8.1. 身份代次：从访问范围到生命周期键

最初，工作流的撤销边界主要由 scope_id 与 control_id 划定。这两个编号能够回答“进程属于哪个访问范围”，但还不能区分同一个有界槽位先后承载的两次运行。

当资源回收、异步事件和槽位复用同时出现时，只比较编号会留下 ABA 风险：旧订阅或旧任务可能把新工作流误认为原来的对象。继续给各模块补局部状态并不能消除这一问题，因为 VFS、IPC、Context 和资源对象需要核对的是同一条生命周期事实。

内核随后拆出生命周期账本，把长期引用统一绑定到带代次的二元键 id+generation。系统一度尝试用分段持久租约维持跨重启连续性；设计复核后又把承诺收窄为仅在单次 boot 内唯一，删除磁盘恢复链路，并规定计数器不回绕、耗尽后关闭接纳。

图3.7给出这次收敛。问题的关键不是再增加一个全局编号，而是让槽位、异步引用和回收屏障都使用同一个带代次的键。

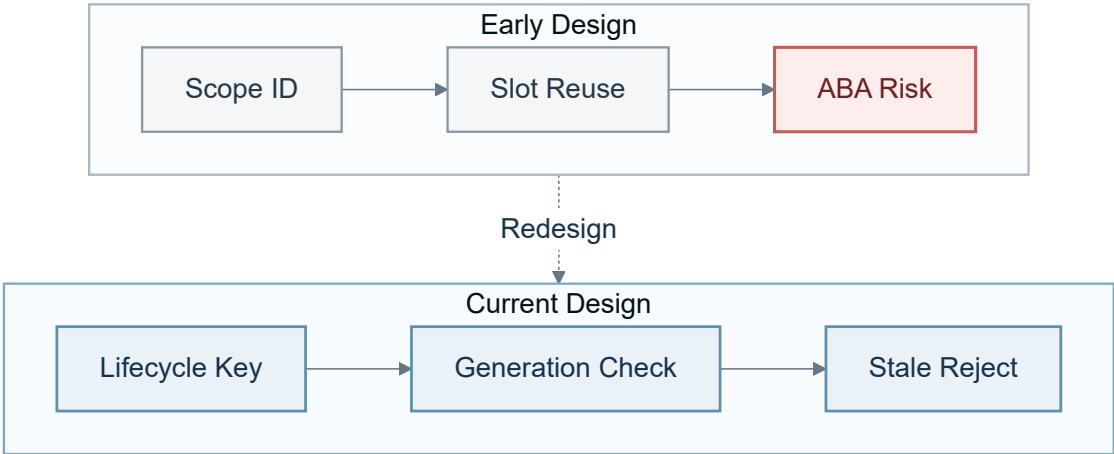


图 3.7: 身份边界由访问范围收敛为带代次的生命周期键

当前查找路径直接由 id 定位固定槽位，再核对 generation。因此，同一编号被复用后，旧键不会因为槽位地址相同而重新生效。

代码 1：生命周期槽位与代次的联合核验

c

```
57 static struct workflow_lifecycle_record *
58 workflow_lifecycle_find_locked(struct workflow_lifecycle_key key)
59 {
60     uint slot;
61     struct workflow_lifecycle_record *record;
62
63     if (!workflow_lifecycle_key_valid(key) ||
64         key.id > WORKFLOW_LIFECYCLE_CAP)
65         return 0;
66     slot = key.id - 1;
67     record = &workflow_lifecycles[slot];
68     if (!record->used || record->generation != key.generation)
69         return 0;
70     return record;
```

回归测试会主动复用相同编号，检查新代次前进，并确认旧键返回 STALE，从而锁定“旧引用不得复活”这一生命周期约束。

3.8.2. 结构化工具：从字段校验到 Execution Contract

初版工具 ABI 使用固定的 `arg0/arg1/payload`，不同工具的规则分散在手写分支中。它足以支撑少量工具，却要求新增工具同时维护说明文字和校验代码，参数错误也难以形成一致的状态语义。

V2 将参数改为有界 `typed key-value`，并由 Registry Schema 统一检查名称、类型、长度、重复键和必填项。到这一阶段，系统已经能够回答“这组参数是否属于该工具”，但仍不能回答“本次尝试是否属于被冻结的执行计划”。

加入依赖、重试和截止时间后，单独的 `tool id` 与参数并不足够。设计复核把边界推进到 V3 Execution Contract，由 Contract `key`、`node`、`attempt`、`schema/input fingerprint`、前驱 Context、`provenance`、`deadline` 和资源额度共同描述一次执行。

当前路径在副作用发生前再次核对运行态声明，如图3.8所示。这样，接纳时有效但在实际执行前已经取消、超时或变旧的请求，不会继续穿过 `effect gate`。

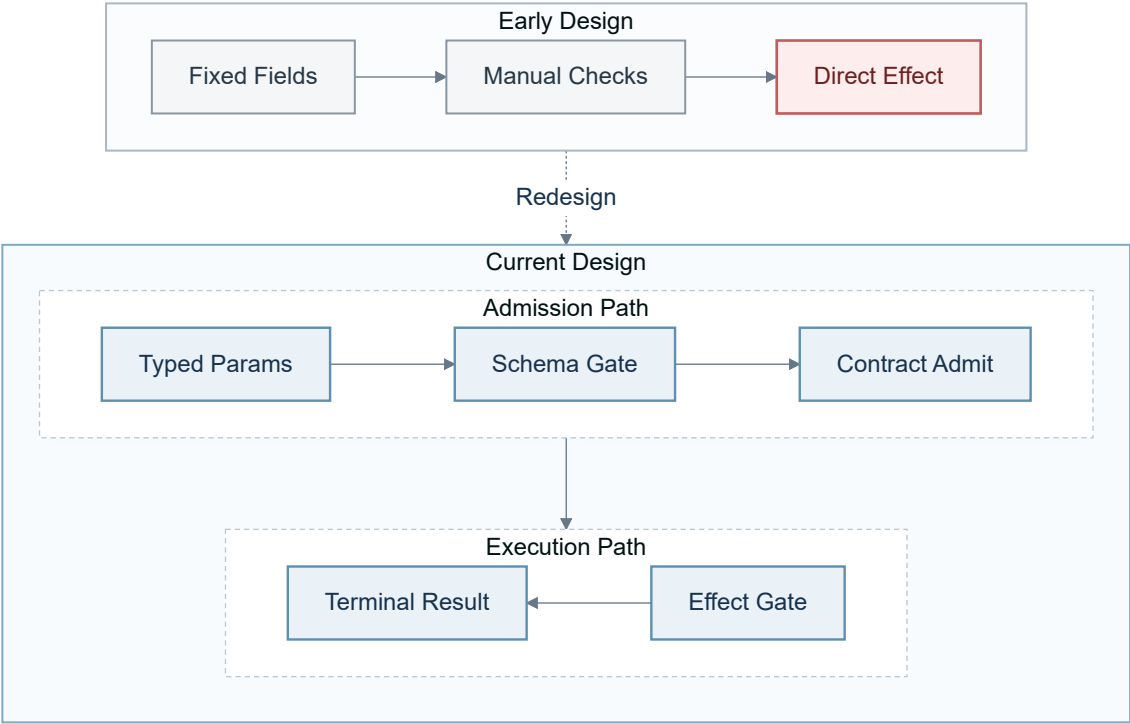


图 3.8: 工具调用从手写字段校验演进为执行约定与副作用门

副作用门先核对 Contract 生命周期和代次，再核对 node、request、attempt 与摘要。任意一项不再匹配时，执行以 STALE 收敛。

代码 2: 副作用前的 Execution Contract 声明复核 c

```
1840     if (!agent_execution_record_matches(record, claim->lifecycle) ||
1841         record->generation != claim->contract_generation ||
1842         claim->node_id >= record->node_count) {
1843         intr_restore(enabled);
1844         return AGENT_EXECUTION_EFFECT_STALE;
1845     }
1846     runtime = &record->runtime[claim->node_id];
1847     if (runtime->state != AGENT_EXECUTION_NODE_RUNNING ||
1848         runtime->request_id != claim->request_id ||
1849         runtime->attempt_id != claim->attempt_id ||
1850         !agent_execution_digest_equal(runtime->request_digest,
1851                                     claim->request_digest)) {
1852         intr_restore(enabled);
1853         return AGENT_EXECUTION_EFFECT_STALE;
```

严格错误矩阵与 24 节点 Contract 回归覆盖 schema 漂移、非法前驱、越权、超时和 replay，并检查资源结算最终回到零。

3.8.3. Context: 从 FIFO 截短到 active path 与 provenance

初版 Context 已经能够按 sequence 保存和查询记录，但查询面对的是物理 FIFO 窗口。最直接的 rollback() 会把最新序号、计数和头指针一起退回，旧分支与当前决策路径无法同时保留，后续追加还可能复用序号。

仅延长 FIFO 不能回答“当前推理链选择了哪一条分支”。系统随后把 `branch_generation`、`path_parent_sequence` 和 `prev_hash` 纳入记录，将物理归档与 `active path` 分开；`successor index` 又把完整路径查询收敛为一次正向扫描。

为了让 Guest 稳定读取，镜像使用奇偶 `publish sequence` 区分写入中与已发布状态。Nexus 则把 `USER`、`TOOL`、`FINAL` 摘要绑定到 `active path`，用该路径重建下一轮消息，而不是维护另一份脱离内核的会话历史。

图3.9保留了回滚后的旧分支。Context rollback 只改变可见路径和后续 `provenance`，不撤销已经发生的文件写入、消息发送或外部服务副作用；需要补偿的外部行为仍由上层工作流负责。

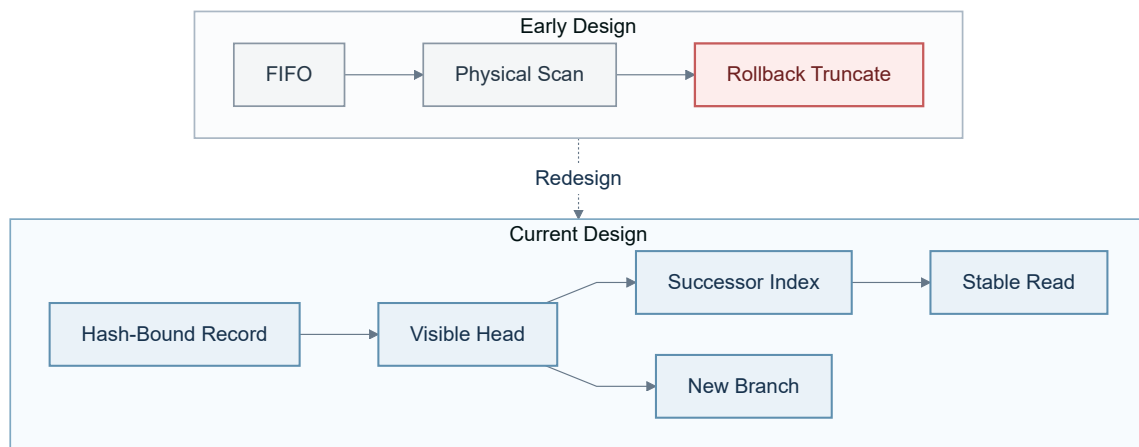


图 3.9: Context 从物理 FIFO 截短演进为可验证的 `active path`

路径读取同时检查窗口边界、记录序号和 `record_hash`，避免被淘汰后复用的槽位或被破坏的记录进入当前路径。

代码 3: `active path` 记录的窗口、序号与哈希核验

c

```

91 static int
92 context_read_sequence(struct proc *p, uint64 sequence,
93                      struct agent_context_record *record)
94 {
95     uint64 slot;
96
97     if (p == 0 || record == 0 || p->context_path_capacity == 0 ||
98         sequence < p->context_path_oldest ||
99         sequence > p->context_path_latest)
100         return -1;
101     slot = (sequence - 1) % p->context_path_capacity;
102     if (agent_context_read_record(p, slot, record) < 0 ||
103         record->sequence != sequence || record->record_hash == 0 ||
104         record->record_hash != agent_context_record_hash(record))
105         return -1;
106     return 0;

```

回归覆盖回滚后归档保留、分支环路、哈希篡改、FIFO 淘汰和 `provenance` 单调性。验证结果表明旧记录在窗口内仍可查、新序号不复用，失败轮次也不会被带入下一轮模型消息。

3.8.4. Catalog: 从自动扫描到 Typed Watch

初版让内核扫描普通目录，并把文件属性写入 `.agentmeta`。后续又加入递归扫描、查询缓存和 `metadata` 恢复，希望自动维持目录、持久记录与索引之间的一致性。

这条路径逐渐同时承担扫描、回写和恢复，恢复进度、回写进度与后台扫描还会相互影响。职责叠加使目录一致性和前台服务都变得难以维护。

重新检查 Agent workflow 真正需要的对象后，当前方案只接纳显式登记的文件，并将 Catalog 明确设为 `volatile`。Live Query 保存完整 `typed predicate`，针对变化前后的集合成员关系产生 `ENTER`、`UPDATE` 或 `LEAVE`。

事件队列出现缺口时，`generation`、完整 `snapshot` 与 `ACK` 共同完成重新同步。图3.10把早期的扫描与字符串 Watch，同当前的显式 Catalog 与集合增量放在同一张图中。

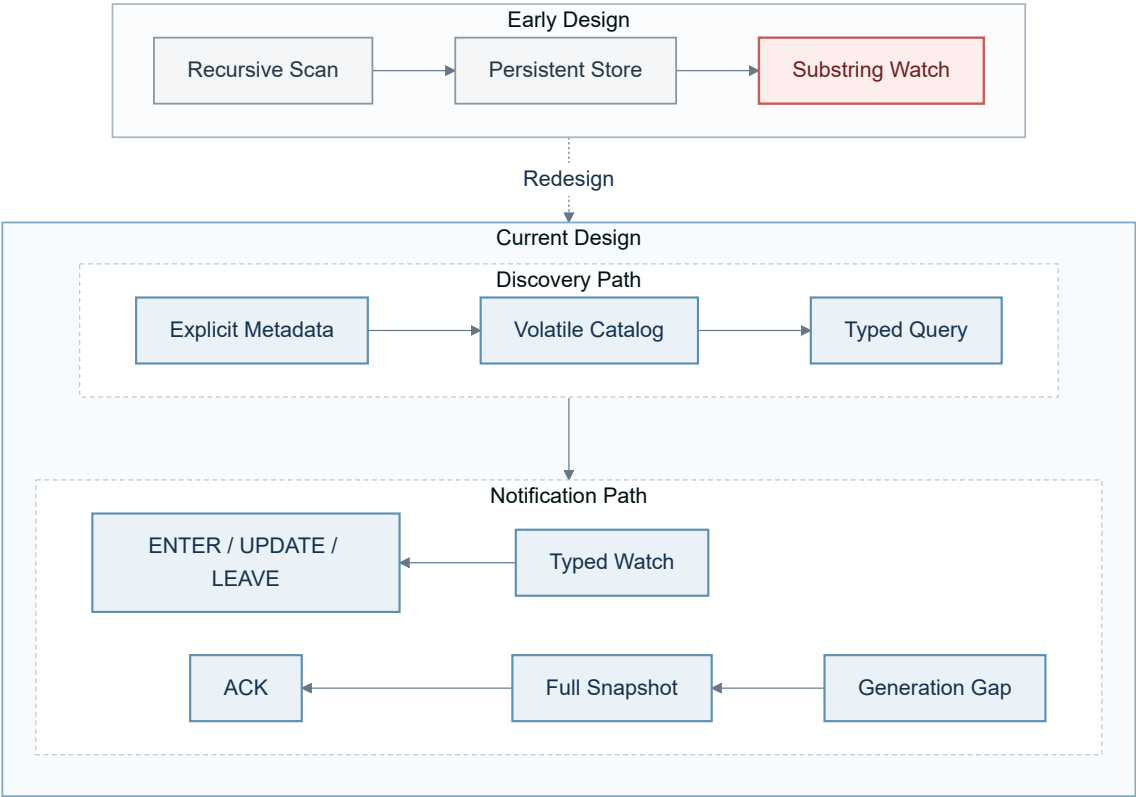


图 3.10: 文件发现从自动持久扫描收敛为显式 Catalog 与 Typed Watch

当前事件类型不是从字符串中猜测，而是由同一谓词在 `before/after` 两个状态上的真值组合直接确定。下面的片段与这一集合语义一一对应。

代码 4: Typed Watch 的 ENTER、UPDATE 与 LEAVE 判定

c

```

572     before_matches = agent_live_query_subscription_matches(
573         subscription, owner_scope, before);
574     after_matches = agent_live_query_subscription_matches(
575         subscription, owner_scope, after);
576     if (!before_matches && after_matches)
577         change = AGENT_LIVE_QUERY_ENTER;
578     else if (before_matches && after_matches)
579         change = AGENT_LIVE_QUERY_UPDATE;
580     else if (before_matches)
581         change = AGENT_LIVE_QUERY_LEAVE;
582     else
583         continue;

```

回归确认普通文件不会自动入表，并覆盖三类事件、缺口恢复与 inode incarnation。Catalog 核心阶段的 16 个配对样本全部更快，中位加速比为 3.118 倍；端到端完整流程则只有 3/16 个样本更快，准备、统计和清理开销抵消了核心查询收益。

3.8.5. 任务委派：从 N1 MESSAGE 到 native Task Channel

初版 Nexus 已能把 child Task 交给 Research 或 System，但任务状态封装在通用的 N1: MESSAGE 中，权威状态仍分散在消息、用户态状态机和等待接口之间。

加入取消与硬截止时间后，Coordinator 还要发送 CANCEL 消息、唤醒目标并过滤迟到结果。继续核对 MESSAGE 编码和 route 只能证明消息来自允许的路径，不能给出 claim 何时生效、引用属于哪一代，以及哪个终态赢得竞争。

最终方案把委派扩展为 native Task Channel 的 pending、claim 和 complete。56 字节 descriptor 只保存 target、任务关联与 capsule 等不可变标识；owner、channel/request/slot generation 则由内核槽位以及 claim/complete 回执绑定。正文和结果保留在 Guest artifact 中，避免把大对象塞回控制描述符。

图3.11展示了从用户态状态过滤到内核终态结算的变化。取消不另造第二条完成记录，而是通过 terminal offer 与 ACK 和 provider 完成路径收敛。

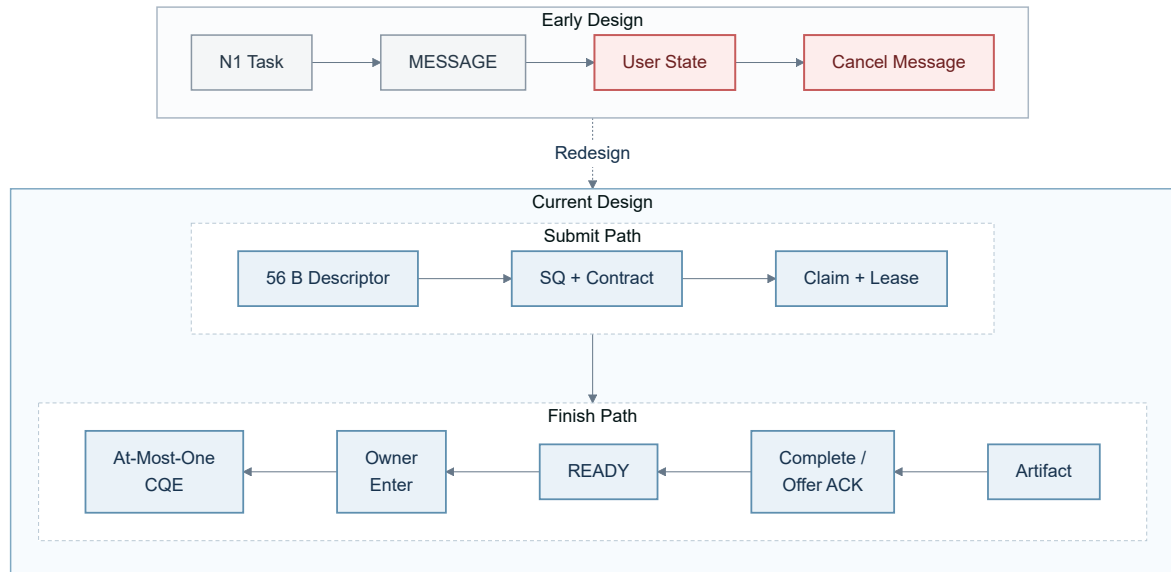


图 3.11: Nexus 委派从 N1 MESSAGE 演进为原生 Task Channel

worker 完成任务时，内核会复核领取回执中的生命周期、进程、线程代次以及完整任务 tuple。下面 21 行比较正是“领取者”和“完成者”不能被拆开的核心约束。

代码 5: delegated Task 完成回执的完整绑定核验

c

```

449 static int
450 agent_task_delegate_receipt_binding_matches(
451     const struct agent_task_delegate_receipt *receipt,
452     const struct proc *worker, const struct thread *thread,
453     const struct agent_task_delegate_complete *complete,
454     struct workflow_lifecycle_key lifecycle)
455 {
456     return receipt != 0 && receipt->valid && worker != 0 && thread != 0 &&
457         complete != 0 &&
458         workflow_lifecycle_key_equal(receipt->lifecycle, lifecycle) &&
459         receipt->worker_pid == worker->pid &&
460         receipt->worker_agent_id == worker->agent_id &&
461         receipt->worker_control_id == worker->agent_control_id &&
462         receipt->worker_thread_generation == thread->identity_generation &&
463         receipt->owner_pid == complete->owner_pid &&
464         receipt->owner_control_id == complete->owner_control_id &&
465         receipt->channel_generation == complete->channel_generation &&
466         receipt->request_id == complete->request_id &&
467         receipt->slot_generation == complete->slot_generation &&
468         receipt->task_id == complete->task_id &&
469         receipt->correlation_id == complete->correlation_id;

```

Task Channel 与 Nexus 回归覆盖 claim 权限、取消和 deadline 竞争、terminal offer/ACK 及资源回收。正常 complete 直接进入 READY；若内核先给出 terminal offer，则须清理并 ACK 最新 offer 后进入 READY，owner 随后的 enter 至多取得一条 terminal CQE。该唯一性由单个 Guest 内核中的通道状态机保证；V1 仍依赖已领取任务的 provider 协作确认终态，无法自动终结永久无响应的执行者。

3.8.6. workflow 服务：从线程份额到 Credit Domain 与 EEVDF

多个 Agent 工作流同时运行时，若沿用 uCore 的线程级取队并把额度分别记在进程或对象上，同一工作流增加线程就可能扩大 CPU 份额，使线程数意外成为调度权重。

资源回收也暴露相同边界：创建中的额度、使用中的额度和退出中的延迟释放若分别结算，工作流结束时难以说明还有多少操作尚未离开。当前实现先把资源收敛到 workflow 的 free/pending/used 账户，再由 Workflow Fence 等待成员、活动操作和延迟回收全部退出。

CPU 服务则在原调度器外增加 workflow EEVDF。系统先以工作流为实体累计 service、vruntime 与 virtual deadline，选中工作流后才从其内部挑选可运行线程，从而把外层公平性与线程个数分开。

图3.12把资源结算与 CPU 服务放在同一条工作流边界上。Credit Domain 控制“还能使用多少”，EEVDF 决定“下一段服务给谁”，两者都引用相同生命周期键。

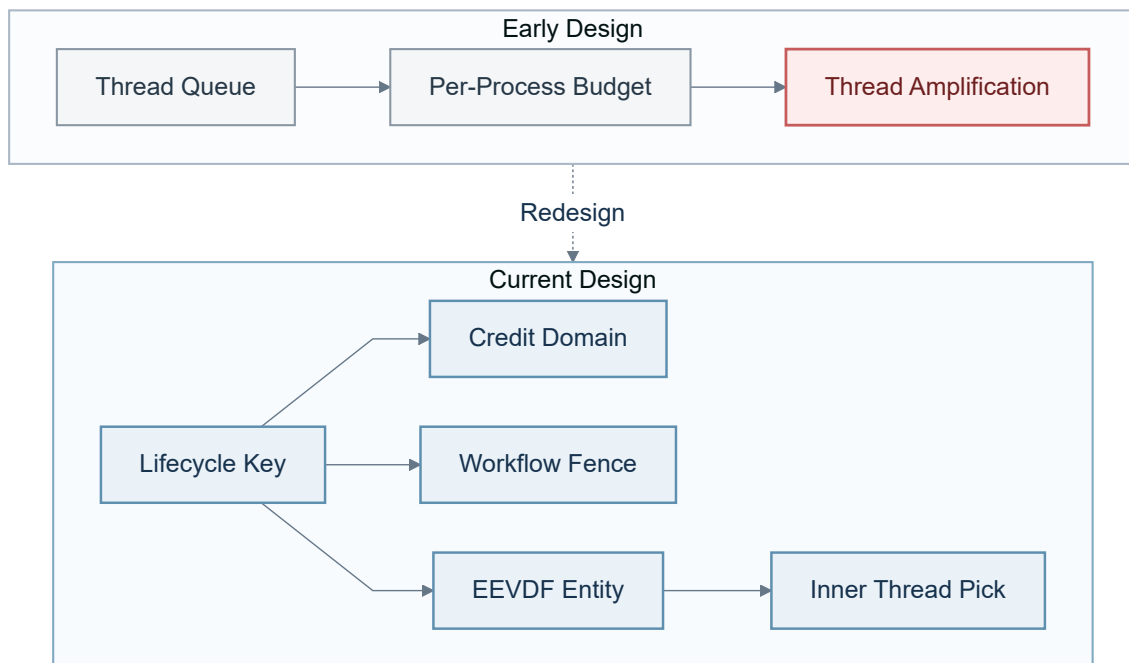


图 3.12: 资源额度与 CPU 服务从线程边界收敛到工作流边界

下面的记账片段给每个工作流实体使用固定权重，并以实际服务量推进 vruntime 和 virtual deadline。新增线程不会在这一层生成新的独立权重。

代码 6: workflow EEVDF 的服务量与虚拟截止时间记账

c

```

1155     entity->service_cycles = workflow_scheduler_counter_add(
1156         entity->service_cycles, service_cycles);
1157     /* All workflow entities have one fixed weight, independent of threads. */
1158     entity->vruntime = workflow_scheduler_add_sat(
1159         entity->vruntime, service_cycles);
1160     if (service_cycles >= entity->remaining_cycles)
1161         entity->remaining_cycles = 0;
1162     else
1163         entity->remaining_cycles -= service_cycles;
1164     entity->virtual_deadline = workflow_scheduler_add_sat(
1165         entity->vruntime, entity->remaining_cycles);

```

六次 QEMU 启动中的 504 次唤醒均落在 0 至 1 tick, 1 至 4 个并发工作流的 Jain 指数中位数均不低于 0.999985。在四个工作流竞争、其中一个使用四线程的场景中, 该工作流份额仍为 26.895%, 高于四等分的 25%, 说明当前实现基本抑制了多线程放大, 但仍存在小幅偏差。

3.8.7. Nexus 边界: Host Broker 与 Guest Authority

Nexus 必须读取 Host 工作区, 而真正的 Agent、Context 和 child Task 又运行在 Guest。初版由 Host 预构建并验证源码快照, 执行 `source_search/source_read`, 再经 MESSAGE 与 N1 把结果交给 Guest worker。这条路径可以工作, 却把检索决策、任务终态和跨轮历史分散在两侧。

继续检查后发现, 路径校验只是其中一环。Host 若继续决定“哪些文件属于本轮证据”或“任务何时完成”, Catalog、Task Channel 与 Context 就无法成为完整的执行链。

当前方案把 Host 收敛为 broker: Host 分页提供 versioned manifest, 并按 Guest 的受限请求返回文件字节; Guest 用 Metadata Catalog 与 Live Query 选择候选, Typed Watch 则在 control stub generation 更新时使旧窗口失效。随后, native Task Channel、artifact 和 active path 完成执行与结算。

Host broker 负责读取受路径、版本和长度约束的工作区字节, 并连接在线 Provider; Guest 则保存文件选择、身份、任务终态和跨轮 Context 等权威语义。

Host 收到模型请求后先按固定上限验证结构, 再对 canonical JSON 计算摘要, 以此绑定所转发请求的内容与版本。

代码 7: Host 对 Guest 请求的有界验证与规范摘要

Python

```

2466     request = relay.validate_guest_request(
2467         provider_payload,
2468         max_output_tokens=self.max_tokens,
2469         max_tool_arguments=self.max_tool_arguments,
2470         max_tool_argument_string_bytes=(
2471             self.max_tool_argument_string_bytes
2472         ),
2473     )
2474     request_sha256 = hashlib.sha256(relay.canonical_json_bytes(request)).hexdigest()

```

将选择路径、失效通知和执行路径放在一起，图3.13更清楚地显示了边界变化：Host 的能力停在版本化 manifest 和受限字节读取，Guest 则保存候选选择、Task 终态和 Context 历史。

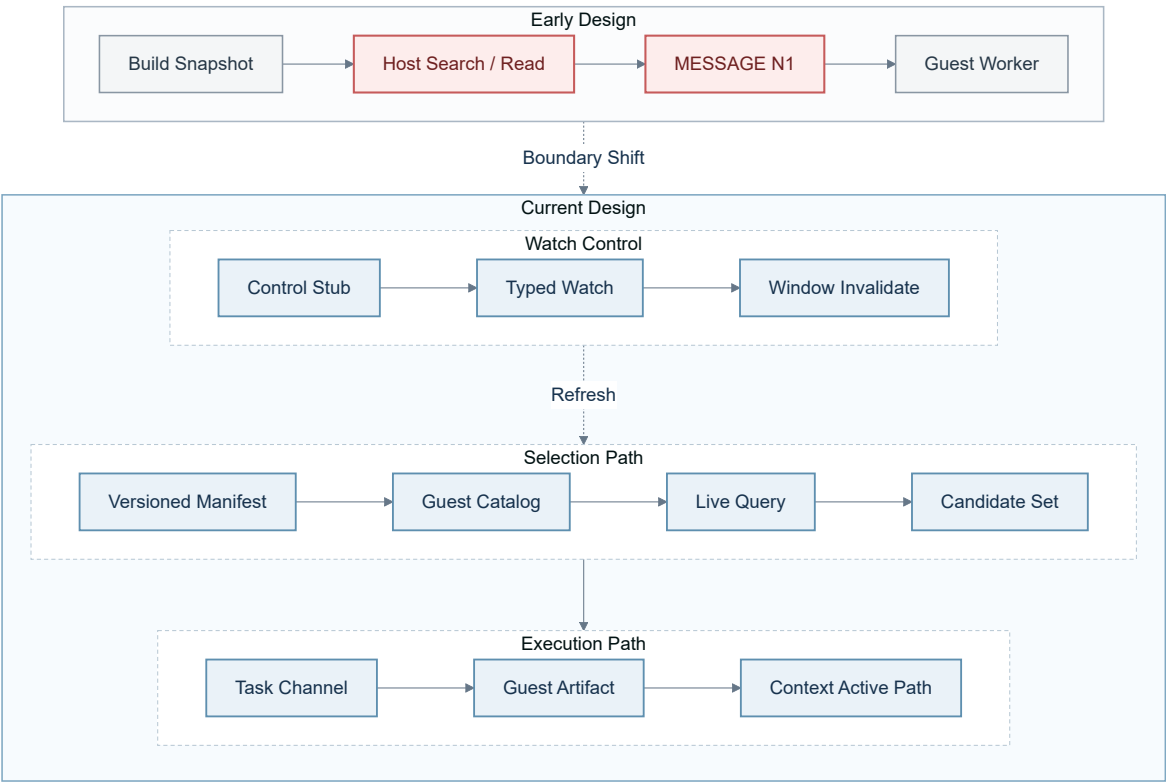


图 3.13: Nexus 从 Host 语义分散收敛为 Guest 权威、Host 代理

固定 QEMU replay 已覆盖 4 个 provider round、2 个真实 child Task 和 2 次版本化 read_file。在线 Provider 与离线 replay 都从 Guest Context 重建模型消息，经过相同的 Host/Guest 执行链。

4. 项目测试

测试先核对接口结构和构建结果，再进入 RISC-V Guest 检查各项内核机制，最后运行控制台、Nexus 和相应的性能负载。功能结果来自真实系统调用输出和协议摘要；性能数据保留逐样本来源，配对比较、分布图和统计表都能追溯到对应的启动记录。

4.1. 功能与性能测试

4.1.1. 测试组织

AgentOS-uCore 的测试从接口结构、内核镜像、Guest 行为和综合场景四个方向交叉检查。结构约定检查核对 UAPI 的尺寸、偏移和模块依赖；当前冻结布局清单覆盖 640 项合约，其中包含 26 项 Tool Registry 和系统调用 567/568 的 delegated Task 结构。交叉编译把真实内核和用户程序链接为 RISC-V 镜像；Guest 场景通过系统调用观察 Agent 身份、Context、VFS、IPC、资源和调度状态；Host 测试则覆盖帧编码、Provider、工作区 broker、Task ledger、artifact 与验证脚本。性能测量直接读取同一 Guest 工作负载产生的时钟和计数。

对应机制	主要程序	观察重点
Agent 进程与 Context 区	agenteval_ucore, agenttrust_ucore, agentscope_ucore, agentfinal_ucore	创建、派生、装入和退出；固定映射、直接读取、回滚、清空与 FIFO 淘汰
结构化接口与工具协议	agenteval_ucore, agenttoolabi_ucore, agentcontract_ucore, agenttask_ucore	26 项目录与 schema；V2/V3；UTF-8/TASK 资源；TASK route、claim/complete、终态确认、唯一 CQE、背压与重新同步
文件查询与实时事件	agenteval_ucore, agentfs_ucore, agentbench_ucore	元数据与摘要；遍历和索引结果一致性；ENTER、UPDATE、LEAVE 与缺口恢复
Agent Loop 与 EEVDF	agenteval_ucore, agentloop_ucore, agentsched_ucore, agent_eevdf_ucore	原子等待、消息与 TIMER 唤醒、超时、线程放大、 workflow 服务份额
综合场景	labdemo_ucore, agentpublish_ucore, agentnexus_ucore	恢复流程；Nexus 三角色、Context active path、Guest Catalog、版本化工作区字节、原生 Task 委派与会话关闭

表 4.1: 系统机制、测试程序与观察重点

自动化检查依次覆盖智能体模块与 ABI 结构约定、Nexus 协议约定和 Host 测试程序。严格的 Nexus ReplayProvider 在全新 QEMU 启动中逐轮核对 Guest 规范请求摘要，并走过模型自主工具选择、Catalog 候选、Task Channel 委派、真实 TOOL artifact、Context 跨轮重建、回滚/清空和完整关闭路径。

```

1 make agent-module-check
2 make agentos-build TOOLPREFIX=riscv64-linux-gnu-
3 make agentos-nexus-check TOOLPREFIX=riscv64-linux-gnu-
4 make local-host-selftests

```

4.1.2. 任务一至任务五的 Guest 综合验收

agenteval_ucore 把赛题任务一至任务五放在同一个 RV64 Guest 中检查。运行命令如下：

```
1 AGENT_TEST_CASE=agenteval_ucore \
2 make agentos-test TOOLPREFIX=riscv64-linux-gnu-
```

这项测试以系统调用后的 Guest 可见状态为判据，五段场景的观察内容见表4.2。

场景	Guest 中可观察的行为
Agent 进程创建与地址空间	Agent 与普通进程可以并存；身份和配额可查询；固定高地址的七页 Context 区中，六页内核镜像保持只读，用户缓存页可写。
内核结构化交互接口	Guest 能列举工具并发起多种带类型信息的调用；未知工具、错误参数类型和小返回缓冲区得到不同的结构化状态。
上下文路径管理	六轮以上连续调用的直接映射读取与系统调用查询一致；回滚和清空改变后续可见路径；连续写入 133 条记录后只保留最新 128 条，系统继续运行且未耗尽内存。
面向 Agent 的文件查询	文件按属性和摘要被找到，不要求调用者预先知道完整路径；遍历与索引返回同一结果；结构化结果可以写入第七页用户缓存。
Agent Loop 内核运行机制	等待线程在无事件时睡眠，消息或 TIMER 到达后继续运行；多个 Agent 同时活动时，工作流调度统计持续推进。

表 4.2: agenteval_ucore 的五段可观察行为

这组综合验收把创建、工具调用、Context、文件查询和 Loop 串在同一次启动中，能够发现单项测试不易暴露的状态衔接问题。例如，Context 映射若只在创建时存在、exec 后丢失，后续的查询缓存场景便无法继续；文件事件若没有正确唤醒等待者，Loop 场景也不会仅凭查询结果“看起来正常”。

4.1.3. 结构化工具与任务通道测量

任务通道测量让紧凑 Batch、单项 V3 和 SQ/CQ 分别执行 16 次等价 ECHO，并轮换三条路径的先后顺序。每条操作序列记录完整墙钟时间，同时核对工具、状态、Context 序号、执行记录票号和结果指纹，确保三条路径完成等价工作。4 次启动共保留 96 条操作序列。

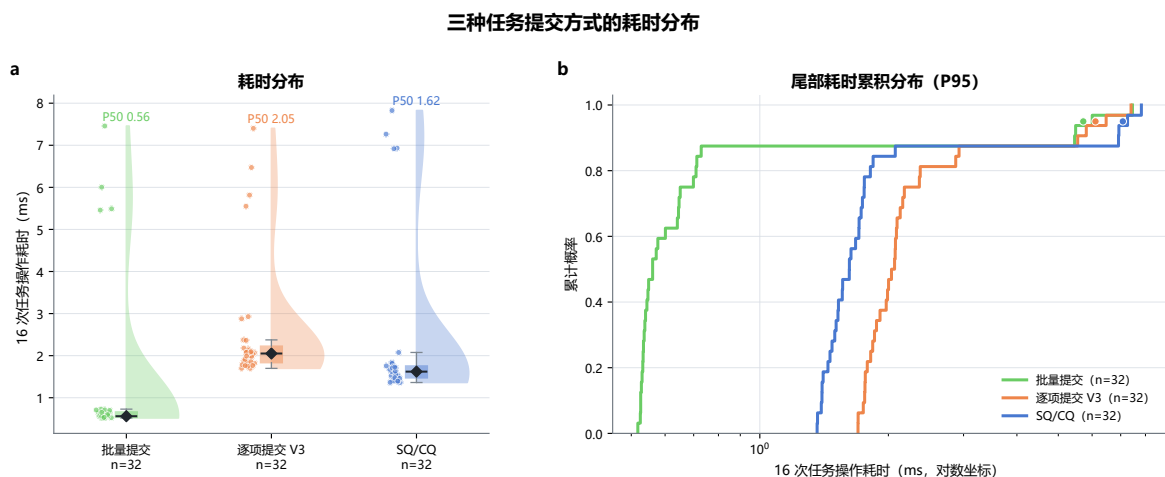


图 4.1: 三种任务提交方式的耗时分布

图4.1中，紧凑 Batch、单项 V3 和 SQ/CQ 完成 16 项等价操作的中位数分别为 561 微秒、2051 微秒和 1620.5 微秒。三条路径分别使用 1、16、2 次系统调用：Batch 在一次进入内顺序执行短操作；V3 为每一项绑定执行约定、节点和尝试号；SQ/CQ 通过共享队列提交并回收结果。每次启动的首轮中位数依次为 5.747、7.098、6.1435 ms，后续轮次降至 0.548、1.9965、1.5615 ms，说明刚进入该路径时的建立和缓存开销会明显抬高延迟，因此统计同时保留首轮与后续轮次。

这一结果表明，短小、同质且需要同步返回的操作适合 Batch。V3 多付出的时间对应 Execution Contract、节点、尝试号和执行记录检查；SQ/CQ 提供有界提交、背压、取消和唯一 terminal CQE。该批数据覆盖本地等价 ECHO，跨 Agent claim/complete、长 Provider 延迟与异步 backlog 另行压测。

4.1.4. 结果文件发布与冲突处理

结果文件发布测试直接在 Guest 中调用 `agent_file_publish()`，同时检查正常写入、同名竞争、错误参数和 Nexus 收敛逻辑。运行命令如下：

```
1 AGENT_TEST_CASE=agentpublish_ucore \
2 make agentos-test TOOLPREFIX=riscv64-linux-gnu-
```

正常路径读取到完整 header、payload 和 EOF。同一访问范围内的两个调用并发发布同名文件时，结果恰好为一个 OK 和一个 DUPLICATE，已经发布的内容没有被覆盖。坏指针、非法路径、错误结构大小、错误版本和非零保留字段均未留下正式目录项；失败与重复请求也没有增加 workflow 资源计数，删除测试生成的两份文件后，块和 inode 计数回到基线。

Nexus 随后按正式路径逐字节回读。已有内容与本次 header、payload 和 EOF 完全一致时，发布结果可以收敛为成功；内容不同则明确拒绝。这组结果表明，未命名 inode 与单次正式目录接入避免了“文件名已经可见、内容尚未写完”的中间状态，同名竞争也没有演变为静默覆盖。失败和重复发布同样沿用 workflow 资源账户的记账与回收路径。

通用 FS epoch 动态回归在 dirty、inflight 和 durable 三种状态下均通过，覆盖有序批提交、fsync、重启重放和既有 power-cut 窗口。作为共用的分配、回收和恢复基础，分配器故障与重启矩阵的 36 个用例也全部得到预期状态；去掉关键 FLUSH 的变异会被测试捕获。结果文件发布沿用这条 checkpoint 与恢复路径。

4.1.5. 文件查询路径的 I/O 观测

查询测量同时记录文件读写、系统调用、扫描记录数、任务完成次数和 workflow 服务次数。图4.2上半部分把核心路径的 bytes_read 和完整流程的全局 I/O 计数分别按核心时间窗内的系统调用次数归一化；下半部分按已完成操作数展示三种任务提交方式的传输成本。

图4.2中各列采用独立量纲，比较单位为同列、同指标和同一统计窗口。原始中位数显示，indexed 路径把 physical reads 从 143.5 降到 58、VirtIO notify 从 247.5 降到 165、requests 从 402.5 降到 332。这与选择性索引先缩小候选集的设计相符，说明查询优化已经减少了读路径上的实际工作量。与此同时，writes 从 213 增到 225.5、flush 从 47 增到 48，目录探测和检查记录也没有下降；完整流程中的写入、刷新和收尾同步仍未变轻，这与图3.5中的端到端结果一致。

Batch、单项 V3、SQ/CQ 分别有 7/32、9/32、6/32 轮跨过 1 个 tick。10 ms tick 用于观

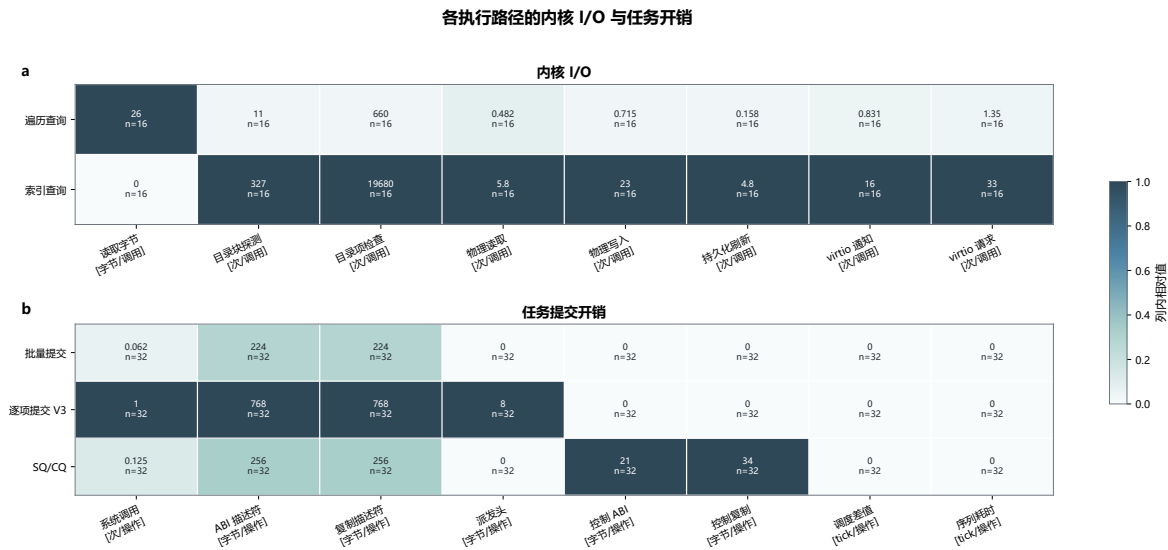


图 4.2: 各查询与任务路径的内核 I/O 观测

察调度节拍，微秒级差异以墙钟测量为准。

4.1.6. 测量数据与可重复性

性能测量使用冻结在源码提交 2b14fb1f74b9bd093e6de939a16554620835699e 的 30 次相互独立全新 QEMU 启动，保存 33 份原始串口输出、19 个 CSV 数据表和 7,498 条结构化记录。每条记录都带有来源文件、来源 SHA-256 和原文序号。同一次启动中的内部配对用于观察相同工作量下的波动；跨启动汇总时，以源启动记录作为独立单位。

对应机制	测量设计	独立单位	样本量
综合恢复与文件查询	16 个 AB/BA 均衡配对启动	启动记录	16 组核心路径和完整流程配对
元数据目录与索引	3 × 4 参数网格，每格 15 个内部配对	源启动记录	180 组查询配对
工具协议与任务通道	4 次启动，8 组轮换顺序	启动记录/区段	96 条 16 次操作序列
Agent Loop 与 EEVDF	6 次启动，并发度为 1–4	启动记录/时间窗和探针	504 条精确唤醒探针

表 4.3: 各内核机制的测量数据构成

数据清单记录源码提交、工具版本、工作量计划、Guest 与构建哈希，并汇总原始输出、CSV 数据表和图像文件哈希；清单中的 75 项均通过哈希校验。图表中的每个点都可以回溯到对应的 Guest 输出。

4.2. 测试路径与结果

测试通过真实 RV64 Guest 调用观察内核对象的状态、返回码、上下文序号、代次和运行统计。功能场景、机制探针和性能测量共用同一构建产物，表4.1中的项目均对应实际执行路径。

4.2.1. Guest 原生工作负载

labdemo_ucore 使用同一组语料完成“定位失败阶段、写回恢复结果、再次确认状态”的业务流程。传统路径逐个打开文件并匹配内容，核心扫描循环如下。

代码 8：传统路径逐文件扫描

c

```

632     for (int i = 0; i < LABDEMO_CORPUS_SIZE; i++) {
633         int fd;
634         ssize_t bytes;
635
636         demo_corpus_name(name, 'c', i);
637         fd = open(name, O_RDONLY);
638         check(fd >= 0, "compat corpus open");
639         memset(buffer, 0, sizeof(buffer));
640         bytes = read(fd, buffer, sizeof(buffer) - 1);
641         check(bytes > 0, "compat corpus read");
642         metrics->bytes_read += (uint64)bytes;
643         metrics->records_examined++;
644         check(close(fd) == 0, "compat corpus close");
645         if (demo_contains(buffer, "stage=align;status=failed") &&
646             demo_contains(buffer, "reason=memory_limit"))
647             found++;
648     }
649     DEMO_DIAG_END("compat", "corpus_scan");
650     check(found == 1, "compat unique failed stage");

```

原生路径把同一筛选条件编码为元数据查询，并检查索引命中数量、阶段和状态。两条路径随后执行相同的恢复写入，并生成相同的结果指纹。

代码 9：原生路径按元数据索引查询

c

```

756     seed_native_workload(&target);
757     memset(&query, 0, sizeof(query));
758     query.flags = AGENT_FILE_QUERY_USE_INDEX;
759     query.max_hits = AGENT_FILE_QUERY_MAX_HITS;
760     strcpy(query.project, DEMO_PROJECT);
761     strcpy(query.run_id, DEMO_BENCH_RUN);
762     strcpy(query.status, "failed");
763     demo_quiescence_fence("native", "CORE_START", 2, &state->core_start);
764     metrics->started_us = demo_now_us();
765     DEMO_DIAG_BEGIN();
766     check(agent_file_query(&query, &result) == 1,
767         "native failed-stage query");
768     DEMO_DIAG_END("native", "failed_index_query");
769     metrics->records_examined += result.scanned_records;
770     check(result.returned == 1 && result.used_index == 1 &&
771         strcmp(result.hits[0].stage, "align") == 0 &&
772         strcmp(result.hits[0].status, "failed") == 0,
773         "native failed-stage result");

```

工作负载同时记录核心查询与完整流程窗口，对比见第3.6节图3.5；图表直接读取当前测量目录的时间与资源数据。

4.2.2. 工具、执行约定与任务通道测试

Nexus Runtime 发起跨 Agent 工具调用时，先发布 Guest capsule artifact，再将目标身份、任务关联和 capsule 句柄写入 56 字节不可变描述符。描述符导入 Task Channel 后，由 `delegate_task` SQE 绑定 Execution Contract 并提交。

代码 10: capsule 描述符的构造与提交

c

```

9143     if (nexus_publish_task_capsule(
9144         capsule_handle, task_id, root_task, task_type, input_handle, 0,
9145         result_handle, objective, argument, target) < 0)
9146         NEXUS_TASK_RETURN (AGENT_STATUS_IO_ERROR);
9147     memset(&descriptor, 0, sizeof(descriptor));
9148     descriptor.version = AGENT_TASK_DELEGATE_DESCRIPTOR_VERSION;
9149     descriptor.size = sizeof(descriptor);
9150     descriptor.target_pid = target_pid;
9151     descriptor.target_agent_id = target->agent_id;
9152     descriptor.task_type = task_type;
9153     descriptor.target_control_id = target->control_id;
9154     descriptor.task_id = task_id;
9155     descriptor.correlation_id = corr_id;
9156     descriptor.parent_task_id = root_task;
9157     descriptor.capsule_handle = capsule_handle;
9158     descriptor.flags = AGENT_TASK_DELEGATE_F_NONE;
9159     if (nexus_task_submit(
9160         &descriptor, &submission, turn_id, request_id) < 0)
9161         NEXUS_TASK_RETURN (AGENT_STATUS_IO_ERROR);

```

目标 Agent 经 claim 取得描述符后，按自身角色读取 capsule，并核对 artifact 类型、来源、任务父子关系、发布状态和所有者。

代码 11: capsule artifact 的来源与任务绑定核验

c

```

8376     if (nexus_read_artifact_for_role(
8377         descriptor->capsule_handle, role, &header, capsule,
8378         sizeof(*capsule), &size) < 0 ||
8379         header.kind != AGENT_NEXUS_ARTIFACT_TASK_CAPSULE ||
8380         header.source != AGENT_NEXUS_SOURCE_MODEL ||
8381         header.task_id != descriptor->task_id ||
8382         header.parent_task_id != descriptor->parent_task_id ||
8383         header.flags != AGENT_NEXUS_ARTIFACT_F_PUBLISHED ||
8384         !nexus_actor_matches_identity(&header.producer,
8385             &nexus_coordinator_identity) ||
8386         !nexus_actor_matches_identity(&header.owner,
8387             &nexus_coordinator_identity) ||
8388         size != sizeof(*capsule) ||

```

artifact 通过核验后，目标 Agent 继续检查 capsule 中的任务类型、输出句柄、字符串边界以及 PID、Agent ID、control ID 和角色绑定。

代码 12: capsule 内容与目标身份核验

c

```

8390     capsule->task_type != descriptor->task_type ||
8391     capsule->result_handle == 0 ||
8392     capsule->objective_length >= sizeof(capsule->objective) ||
8393     capsule->objective[capsule->objective_length] != 0 ||
8394     capsule->argument_length >= sizeof(capsule->argument) ||
8395     capsule->argument[capsule->argument_length] != 0 ||
8396     capsule->target.pid != (uint) getpid() ||
8397     capsule->target.agent_id != (uint) info->agent_id ||
8398     capsule->target.control_id != descriptor->target_control_id ||
8399     capsule->target.kernel_role != (uint) role ||
8400     capsule->target.product_role != nexus_product_role(role))
8401     return AGENT_STATUS_BAD_PARAM;

```

这条端到端路径分别观察用户态编码失败、内核 schema/route 拒绝、任务终态和工具副作用失败。执行约定测试还覆盖过期生命周期、过期约定、未知节点、工具不匹配、截止时间与依赖终止状态（terminal state）。副作用前检查产生的完成记录关联执行记录票号与输出来源；直接调用或 V3 调用在关键记录不足时返回 NO_SPACE/RETRY，使拒绝和取消结果能够回溯到上下文与执行记录。

任务通道在提交后维护已接纳和运行状态、生命周期引用以及回调引用；硬截止时间触发过期，完成或拒绝进入统一的完成队列。状态机测试核对生命周期引用完整性，并确认过期任务按照终态规则结算。

4.2.3. Nexus 会话与 Host Relay

Nexus 使用带序号和摘要的帧，把 Guest 的会话、工具事件、审批和运行指标接入 Host Relay。用户态解码器校验会话、序号、类别、长度和载荷摘要，中继程序再完成 Provider 适配和本地端点转发。网络与模型协议位于 Host 侧，智能体身份、工具权限和任务状态由 Guest 与内核验证。

测试运行还检查关闭、取消和过期后的清理过程。Host Relay 可以请求停止，Guest 把等待和工具事件转换为可观察的完成记录；内核则利用生命周期代次，阻止回收后的迟到 Task completion、工作区结果或订阅引用落到新的工作流。

4.2.4. 失效状态与恢复分支

测试主动构造过期代次、错误 schema、资源不足、SQ/CQ 协议失步、实时查询缺口和失效 artifact 句柄。各接口分别返回 STALE、DENIED、NO_SPACE、RETRY 等状态，并保持尚未提交的副作用不可见。任务通道在协议失步后要求显式 RESYNC；取消测试由同一活动生命周期内的 controller 使用 AGENT_TASK_DELEGATE_COMPLETE_F_REQUEST_CANCEL 和准确的 owner/channel/request/slot/task/correlation tuple 发起，同时检查 ORCHESTRATE、WAIT_CANCEL 与 TASK route。返回 OK 只确认 control request 已接收；任务已经 CLAIMED 时，provider 仍须清理预绑定结果并确认最新终态 offer。工作区 generation 变化后，Guest 通过 Typed Watch 使旧 Catalog 窗口失效并从 manifest cursor 0 重建。

5. 总结与展望

5.1. 阶段性结论

本项目在 RISC-V uCore 中实现了内核中的智能体运行机制。AgentOS-uCore 以现有的进程、页表、VFS、系统调用和调度器为基础，把身份、工具、上下文、文件查询、事件等待、资源和调度纳入同一工作流。六组机制让智能体在发生副作用时都具有明确的身份、能力位、访问范围、代次、数据来源和资源账户。

项目取得了三方面进展。第一，Agent 已从应用层标签变为内核能够识别的进程身份，普通进程和 Agent 进程可以在同一个 uCore 中共存，高频 Context 数据也能从固定用户地址直接读取。第二，工具调用协议、V2/V3、上下文路径、实时查询、原生 Task Channel、资源账户和 EEVDF 贯通了 Agent 从调用、等待到取得资源服务的主要过程。第三，控制台和 Nexus 把这些内核机制用于科研恢复场景和三角色常驻协作；Nexus 直接复用 Guest Context、Metadata Catalog、Live Query、Typed Watch 与 delegated Task Channel，使确定性测试与模型驱动交互走同一条 Guest 路径。

实验展示了各条路径的性能特征。索引核心路径在 16 个独立配对中均取得更低延迟，配对加速比中位数为 3.118 倍；12 组文件目录网格的中位加速比介于 1.164–2.808 倍；504 条 EEVDF 唤醒探针均在 0–1 个 tick 内得到服务。端到端窗口中，索引路径相对遍历路径的中位差值为 +13.452 ms，说明核心窗口外的总开销抵消了索引收益。

5.2. 当前技术限制

当前测试环境是 RV64 QEMU 单 Hart，工作流 EEVDF 的实体迁移和多核竞争尚未测量。实时查询单次最多返回 8 个结果，ABI 还没有分页游标；事件队列、Context 路径、任务通道和调度实体表也采用固定容量。现有负载以短时、可重复的 Guest 场景为主，尚未覆盖长 Provider 延迟下的大规模异步 backlog。调度观测使用 10 ms 的 tick，适合判断唤醒是否跨越节拍，不足以分辨微秒级差异。

5.3. 后续工作

下一阶段将补充分页查询和长时间压力场景，重点观察长 Provider 延迟、任务积压、订阅事件密集到达时的背压行为；随后把工作流 EEVDF 扩展到 SMP 和真实 RISC-V 硬件，分析实体迁移、唤醒和资源记账。性能测量还会进一步拆分场景准备、查询、写入、刷新和清理阶段，并使用更细粒度的时钟定位完整流程中的额外开销。

5.4. 比赛收获

比赛过程中，最明显的变化并不是代码量增加了多少，而是我们对“操作系统应该负责什么”的判断逐渐具体。项目早期很容易把 AgentOS 理解成给 uCore 增加若干面向智能体的接口；真正把模型请求、Guest 应用、内核对象和 Host 服务串起来以后，我们才更清楚地看到，内核不应接管模型策略，而应负责那些必须统一裁决的部分：执行者身份、内

核可见的工具副作用、对象代次、任务终态和工作流资源账户。把策略留在用户态与 Host，把共同约束放到 uCore 内核执行层，这一边界判断比单独实现某个系统调用更有价值。

我们也重新认识了“功能跑通”和“系统闭环”的差别。系统闭环要求模块在取消、超时、进程退出、代次复用和资源耗尽时仍然保持一致。组合测试迫使我们把问题逐项说清：Task Channel 的取消和迟到完成只能产生一个 terminal CQE；Metadata Catalog 与 Typed Watch 必须让旧 generation 的窗口失效；Workflow Fence 和退出路径则要在关闭时回收对象与资源。做过这些检查以后，我们逐渐习惯在跨层接口中先说明谁保存权威状态、谁传递引用、谁负责最后校验，再开始写实现。

调试方法也发生了变化。QEMU 串口、固定 ReplayProvider、分配器故障与重启矩阵，以及 Context、状态码和资源计数，让我们能够把模型输出的随机性与内核、协议和文件系统问题分开：先把失败稳定复现，再沿 Host、Guest、系统调用和内核状态逐层缩小范围。测试会检查真实调用路径、Context 序号、失败后的继续运行，以及块、inode 和工作流资源计数是否回到基线。测量显示索引核心阶段更快，但完整流程的收益并不稳定；我们据此分别报告两个测量窗口，并继续定位准备、写入与清理阶段的额外开销。

协作方面，我们体会最深的是，系统项目不能只依靠口头同步。共享 ABI 的结构大小和字段偏移、可以直接运行的构建命令、失败时能够定位到源码的测试，以及随实现同步更新的设计文档，都是协作接口的一部分。它们让内核、Guest 和 Host 的修改能够被另一侧独立复核，也让后来发现的边界问题不必从头猜测。回头看，这次比赛真正改变的是我们的工作顺序：先写清可信边界和状态权威，再用能够复现、能够归因的实验检验实现；异常路径、文档和复现命令也不再只是收尾材料，而成为功能交付的一部分。对我们而言，这些方法比某一个接口或某一次性能结果更值得保留。

6. 参考文献

- [1] Chloe Autio, Reva Schwartz, Jesse Dunietz, Shomik Jain, Martin Stanley, Elham Tabassi, Patrick Hall, and Kamie Roberts. Artificial intelligence risk management framework: Generative artificial intelligence profile. NIST AI 600-1, National Institute of Standards and Technology, Gaithersburg, MD, July 2024. URL <https://doi.org/10.6028/NIST.AI.600-1>.
- [2] OWASP Gen AI Security Project. OWASP Top 10 for LLM Applications 2025. Version 2025, released November 18, 2024, November 2024. URL <https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/>. Licensed under CC BY-SA 4.0.
- [3] OWASP Gen AI Security Project, Agentic Security Initiative. Agentic AI – threats and mitigations. Version 1.1, December 2025. URL <https://genai.owasp.org/resource/agentic-ai-threats-and-mitigations/>. Licensed under CC BY-SA 4.0.
- [4] Model Context Protocol. Mcp specification 2026-07-28. <https://modelcontextprotocol.io/specification/2026-07-28>, 2026.
- [5] A2A Project. A2a protocol specification v1. <https://a2a-protocol.org/latest/specification/>, 2026.
- [6] LearningOS Community. ucore-tutorial-code-2025s. <https://github.com/LearningOS/uCore-Tutorial-Code-2025S>, 2025. AgentOS-uCore 的教学内核基础.